Process Concept
— Chapter 3

叶冠华

g.ye@bupt.edu.cn

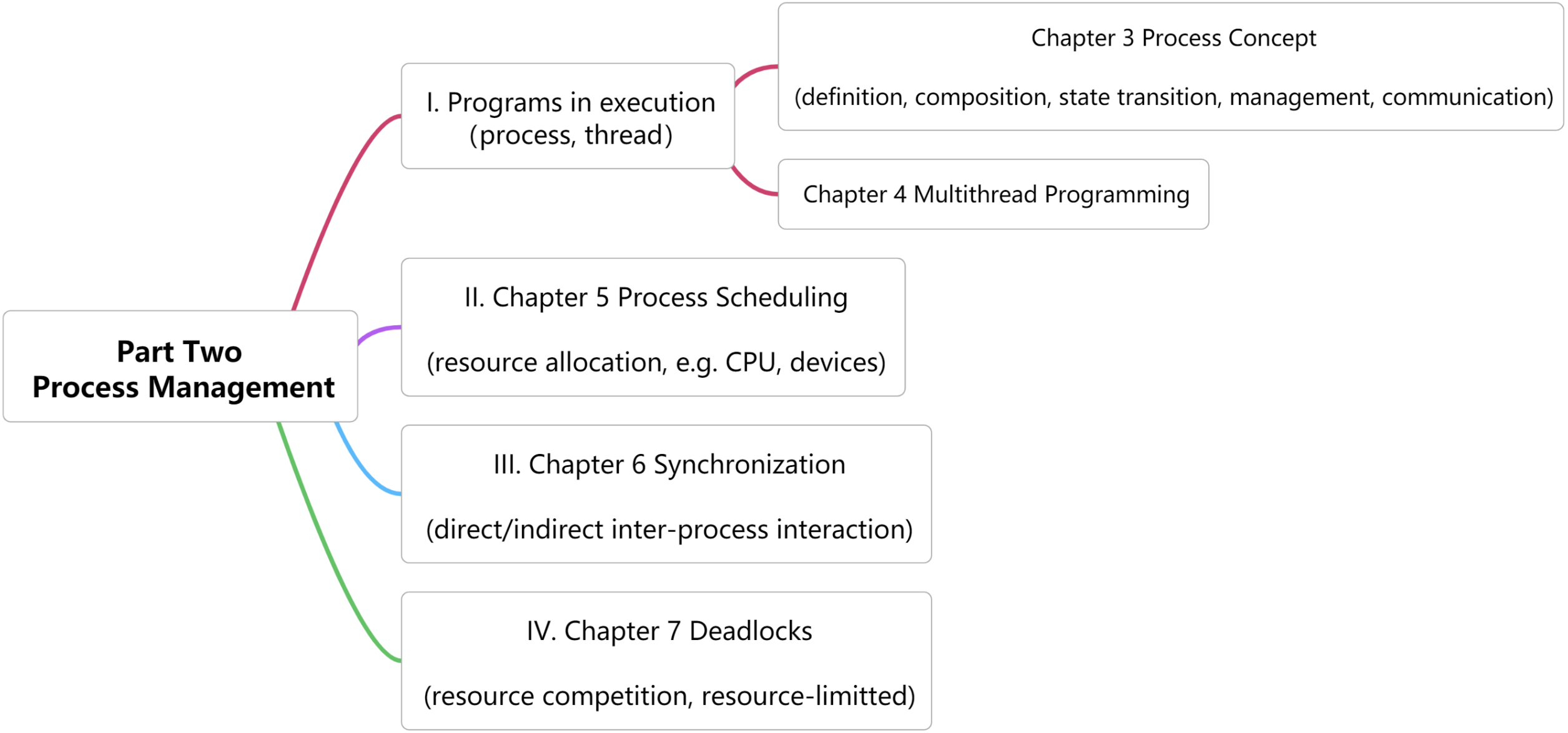
School of Computer Science (National Pilot Software Engineering School)

北京郵電大學

Review

- 系统调用
  - 操作系统提供给应用程序的接口（公共子程序），保证稳定性和安全性
  - 功能：设备管理、文件管理、进程控制、进程通信、内存管理
  - 处理过程：传参→内核态→保存CPU现场→子程序→恢复CPU现场→用户态
- 操作系统结构
  - 分层法：高层->低层单向依赖；易调试，但是低效且不灵活
  - 模块化：按照功能分块，用接口通信；可维护性强，但接口设计复杂、调试困难
  - 微内核：相对于宏内核性能降低，但有高可扩展性、可移植性、可靠性和安全性
- 操作系统的引导
  - CPU激活并跳转到BIOS(存放于ROM)→ 硬件自检(POST)→按启动顺序(Boot Sequenc)e找到并加载主引导记录(MBR) →加载分区引导记录(PBR) →加载启动管理器(Boot Manager)→加载操作系统到RAM

Part Two Process Management

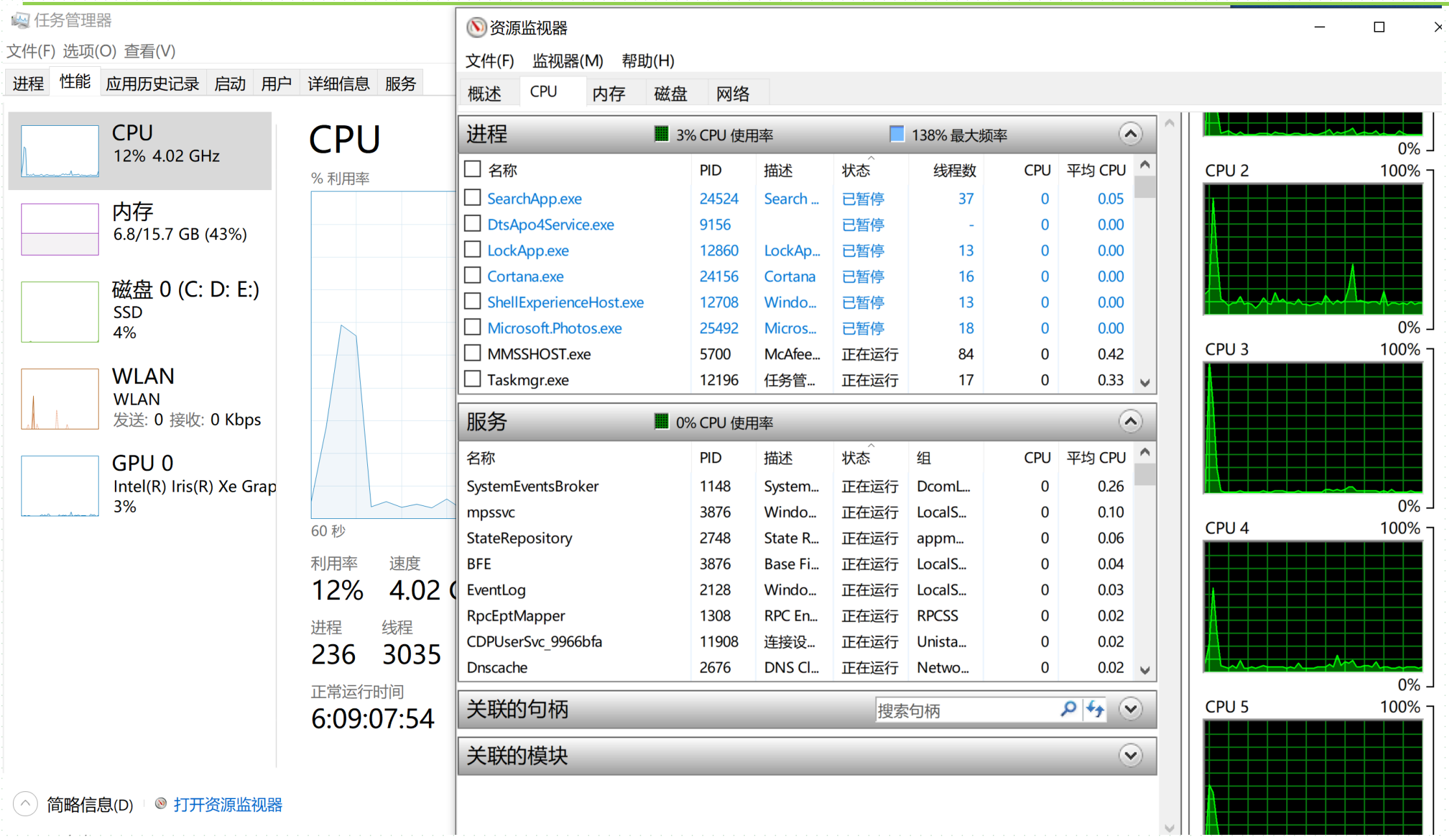


Terminology

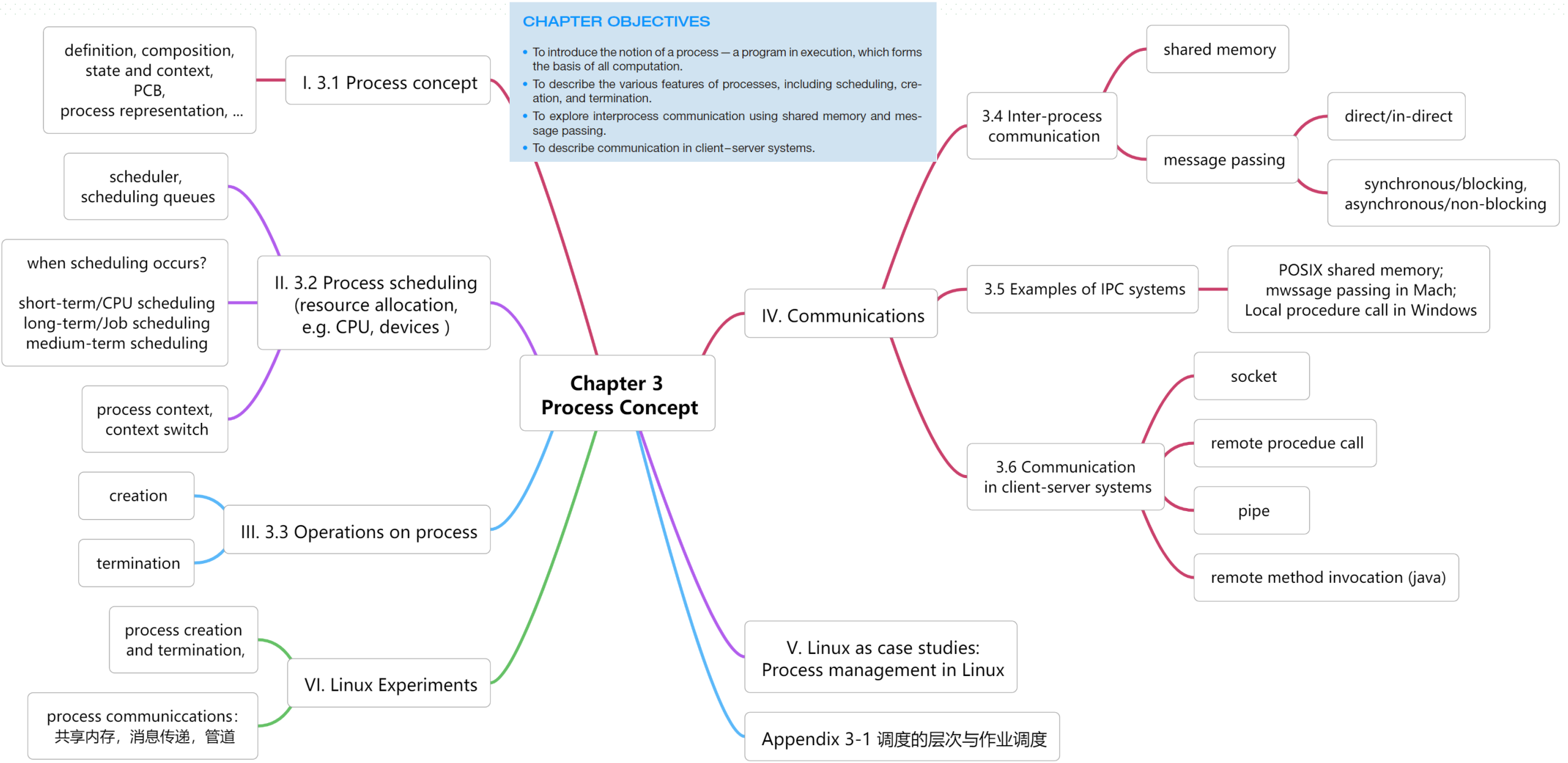
degree of multiprogramming concurrency, parallelism	多道程序设计粒度,并发, 并行
synchronization, mutually exclusive/exclusion	同步, 互斥
process context, CPU switch, process migration	进程上下文, CPU切换, 进程迁移
text section, data section program counter(PC), heap, stack	文本段, 数据段, 程序计数器, 堆, 栈
short-term schedule, long-term scheduler, medium-term scheduling, swapping	短期调度, 中期调度, 长期调度, 交换/倒换
I/O-bound process, I/O-intensive process	I/O受限进程, I/O密集型进程
CPU-bound process, CPU-intensive process	CPU受限进程, CPU密集型进程

abort, cascading	夭折,级联
zombie process, orphan process	僵尸进程, 孤儿进程
Interprocess Communication IPC, shared memory, message passing	进程间通信IPC, 共享内存, 消息传递
blocking, synchronous, asynchronous rendezvous	阻塞, 同步, 异步, 会合
sockets, Remote Procedure Call(RPC), pipes, Remote Method Invocation, marshalls	套接字, 远程过程调用, 管道, 远程方法调用, 封装

Processes in Windows Task Manager



Chapter 3 Process Concept



3.1 Process Concept

- Programs vs processes
- In multiprogramming/多道程序设计 systems and time-shared systems
  - more than one programs, i.e. jobs, user programs or OS programs, execute concurrently
    - Jobs (作业) in batch systems, tasks (任务) in time-shared systems
  - one program may execute several times, processing different data in each time
    - e.g. several opened PPT documents, one PowerPoint program POWERPNT.exe
  - constraints among programs
    - direct constraints such as synchronization/同步 among steps of process execution
    - resources constraints such as mutually exclusive/互斥 usage of I/O devices
  - different programs' execution phases

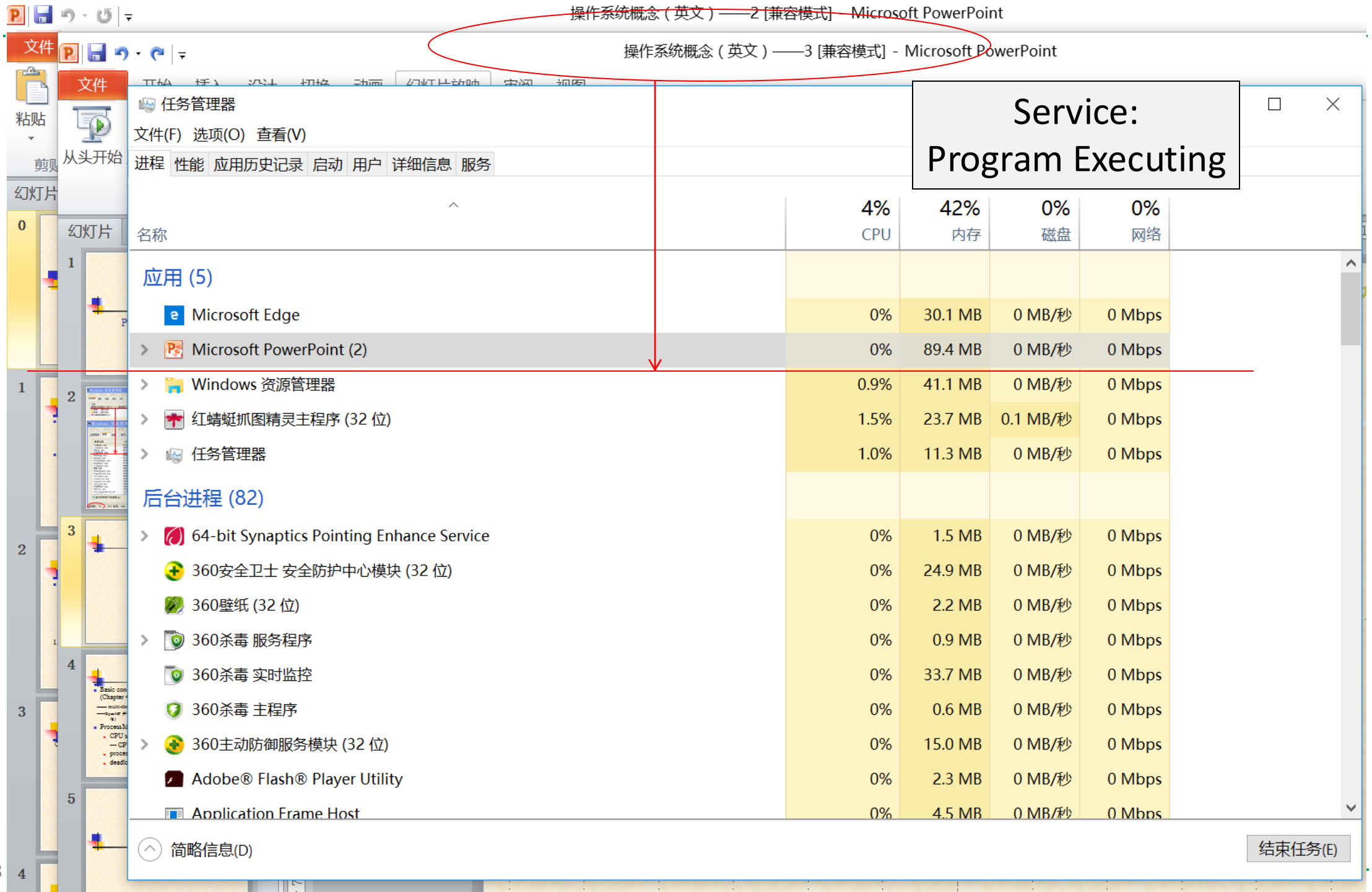
    e.g. start, execute, halt, end → process states

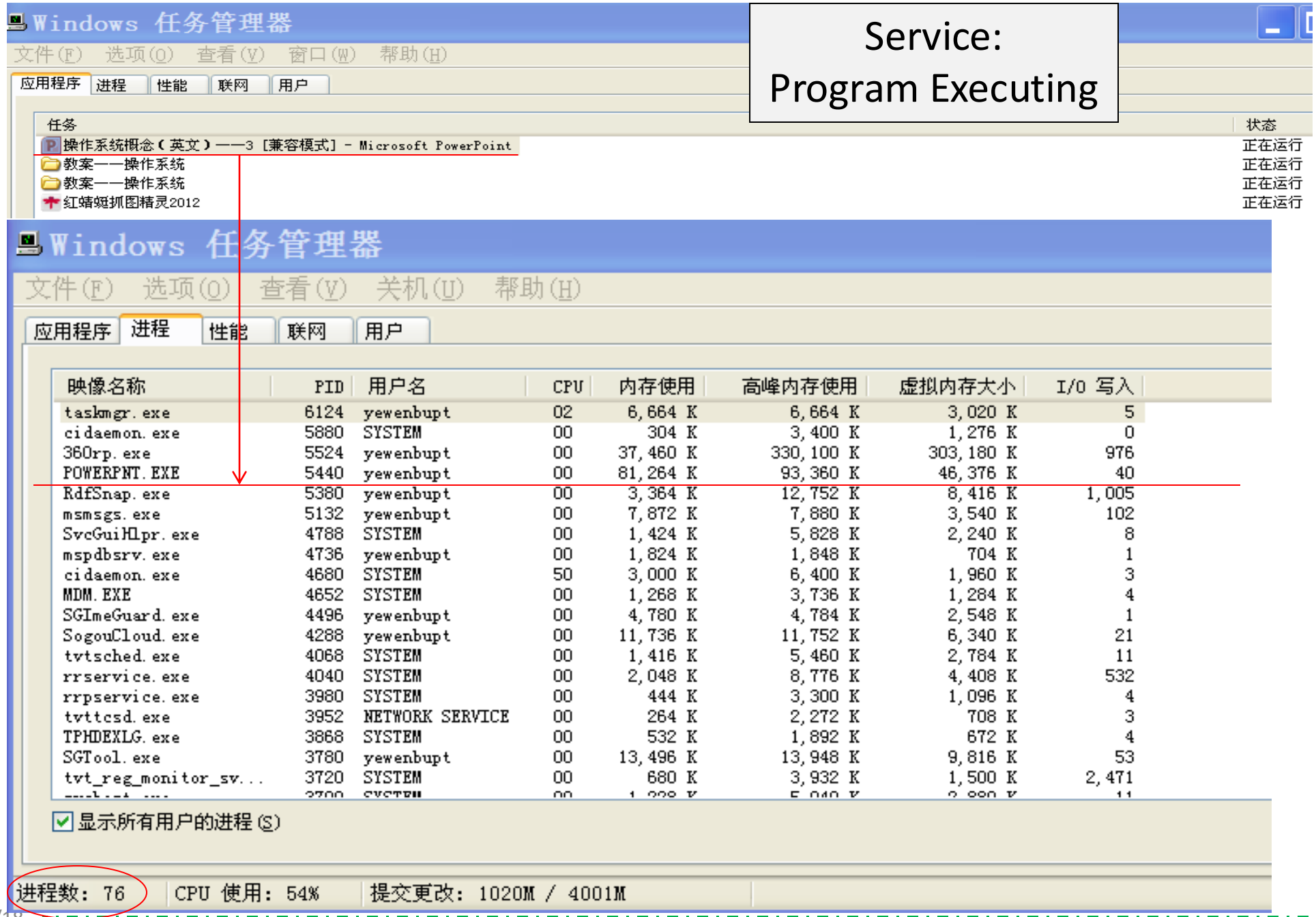
- Processes are thus used for describing of concurrent executing of programs

Process Concept

- Processes are thus used for describing of concurrent executing of programs
- **Process**
  - a program in execution
  - process execution must progress in sequential fashion
    - i.e. process states, such as ready, running, waiting,…
  - OS allocates CPU, memory, devices to process

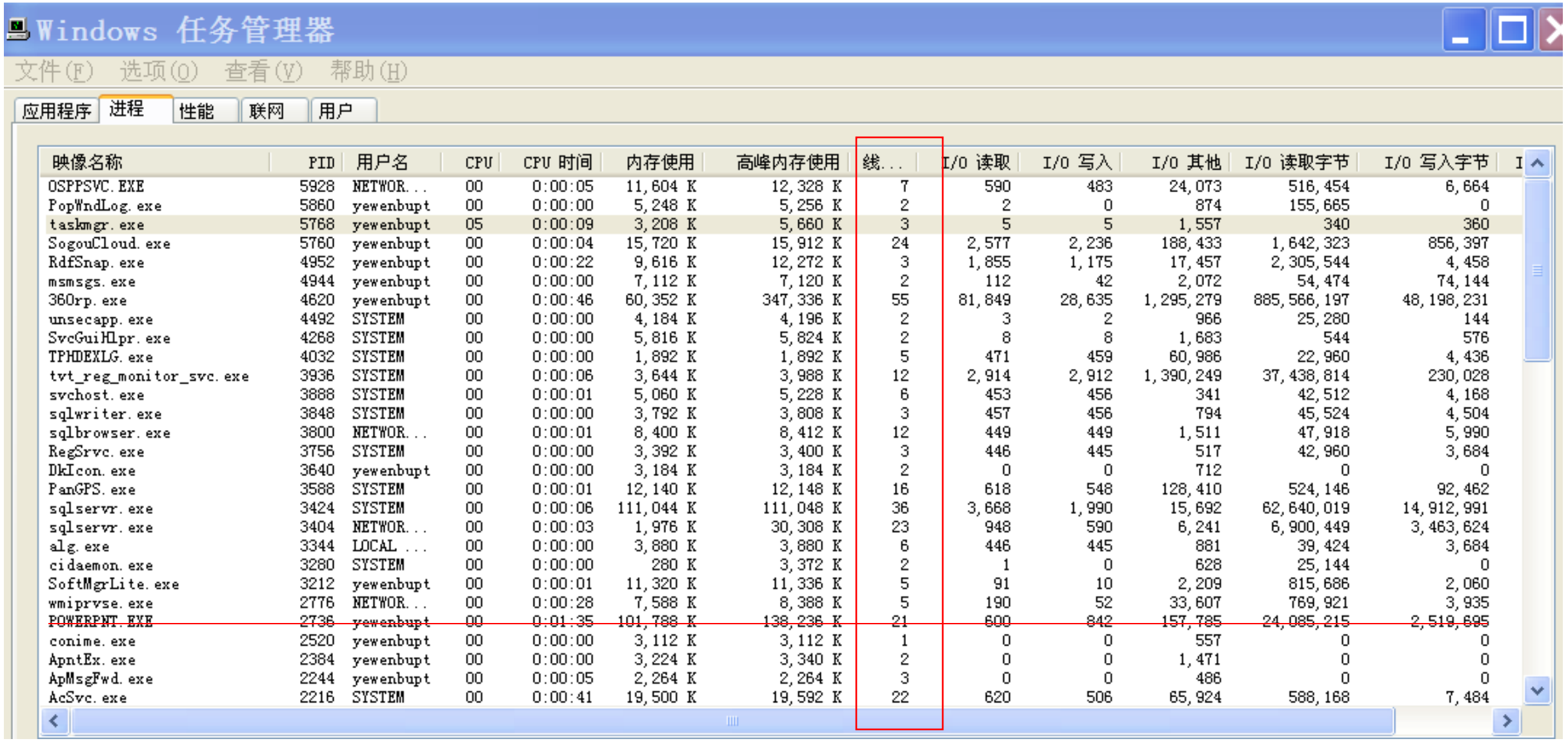
- Generally, process is identified and managed by OS via a **process identifier** (**pid**)
  - refer to next slide
- Textbook uses the terms ***job*** and ***process*** almost interchangeably


操作系统概念（英文）——2 [兼容模式] Microsoft PowerPoint
操作系统概念（英文）——3 [兼容模式] - Microsoft PowerPoint
Service:
Program Executing
任务管理器
文件(F) 选项(O) 查看(V)
进程 性能 应用历史记录 启动 用户 详细信息 服务
名称
4% CPU
42% 内存
0% 磁盘
0% 网络
应用 (5)
Microsoft Edge 0% 30.1 MB 0 MB/秒 0 Mbps
Microsoft PowerPoint (2) 0% 89.4 MB 0 MB/秒 0 Mbps
Windows 资源管理器 0.9% 41.1 MB 0 MB/秒 0 Mbps
红蜻蜓抓图精灵主程序 (32 位) 1.5% 23.7 MB 0.1 MB/秒 0 Mbps
任务管理器 1.0% 11.3 MB 0 MB/秒 0 Mbps
后台进程 (82)
64-bit Synaptics Pointing Enhance Service 0% 1.5 MB 0 MB/秒 0 Mbps
360安全卫士 安全防护中心模块 (32 位) 0% 24.9 MB 0 MB/秒 0 Mbps
360壁纸 (32 位) 0% 2.2 MB 0 MB/秒 0 Mbps
360杀毒 服务程序 0% 0.9 MB 0 MB/秒 0 Mbps
360杀毒 实时监控 0% 33.7 MB 0 MB/秒 0 Mbps
360杀毒 主程序 0% 0.6 MB 0 MB/秒 0 Mbps
360主动防御服务模块 (32 位) 0% 15.0 MB 0 MB/秒 0 Mbps
Adobe® Flash® Player Utility 0% 2.3 MB 0 MB/秒 0 Mbps
Application Frame Host 0% 4.5 MB 0 MB/秒 0 Mbps
简略信息(D)
结束任务(E)



Service:
Program Executing

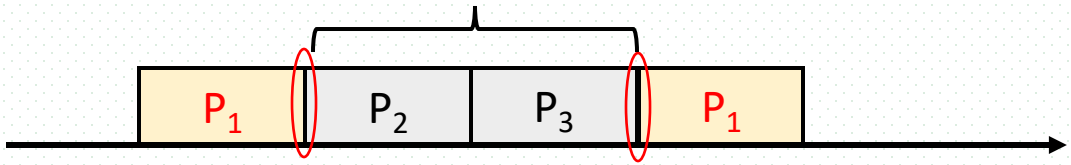
- Resources allocated to programs in execution



映像名称	PID	用户名	CPU	CPU 时间	内存使用	高峰内存使用	线...	I/O 读取	I/O 写入	I/O 其他	I/O 读取字节	I/O 写入字节	I
OSPPSVC.EXE	5928	NETWOR...	00	0:00:05	11,604 K	12,328 K	7	590	483	24,073	516,454	6,664	
PopWndLog.exe	5860	yewenbupt	00	0:00:00	5,248 K	5,256 K	2	2	0	874	155,665	0	
taskmgr.exe	5768	yewenbupt	05	0:00:09	3,208 K	5,660 K	3	5	5	1,557	340	360	
SogouCloud.exe	5760	yewenbupt	00	0:00:04	15,720 K	15,912 K	24	2,577	2,236	188,433	1,642,323	856,397	
RdfSnap.exe	4952	yewenbupt	00	0:00:22	9,616 K	12,272 K	3	1,855	1,175	17,457	2,305,544	4,458	
msmsgs.exe	4944	yewenbupt	00	0:00:00	7,112 K	7,120 K	2	112	42	2,072	54,474	74,144	
360rp.exe	4620	yewenbupt	00	0:00:46	60,352 K	347,336 K	55	81,849	28,635	1,295,279	885,566,197	48,198,231	
unsecapp.exe	4492	SYSTEM	00	0:00:00	4,184 K	4,196 K	2	3	2	966	25,280	144	
SvcGuiHlpr.exe	4268	SYSTEM	00	0:00:00	5,816 K	5,824 K	2	8	8	1,683	544	576	
TPHDEXLG.exe	4032	SYSTEM	00	0:00:00	1,892 K	1,892 K	5	471	459	60,986	22,960	4,436	
tvt_reg_monitor_svc.exe	3936	SYSTEM	00	0:00:06	3,644 K	3,988 K	12	2,914	2,912	1,390,249	37,438,814	230,028	
svchost.exe	3888	SYSTEM	00	0:00:01	5,060 K	5,228 K	6	453	456	341	42,512	4,168	
sqlwriter.exe	3848	SYSTEM	00	0:00:00	3,792 K	3,808 K	3	457	456	794	45,524	4,504	
sqlbrowser.exe	3800	NETWOR...	00	0:00:01	8,400 K	8,412 K	12	449	449	1,511	47,918	5,990	
RegSrvc.exe	3756	SYSTEM	00	0:00:00	3,392 K	3,400 K	3	446	445	517	42,960	3,684	
DkIcon.exe	3640	yewenbupt	00	0:00:00	3,184 K	3,184 K	2	0	0	712	0	0	
PanGPS.exe	3588	SYSTEM	00	0:00:01	12,140 K	12,148 K	16	618	548	128,410	524,146	92,462	
sqlservr.exe	3424	SYSTEM	00	0:00:06	111,044 K	111,048 K	36	3,668	1,990	15,692	62,640,019	14,912,991	
sqlservr.exe	3404	NETWOR...	00	0:00:03	1,976 K	30,308 K	23	948	590	6,241	6,900,449	3,463,624	
alg.exe	3344	LOCAL ...	00	0:00:00	3,880 K	3,880 K	6	446	445	881	39,424	3,684	
cidaemon.exe	3280	SYSTEM	00	0:00:00	280 K	3,372 K	2	1	0	628	25,144	0	
SoftMgrLite.exe	3212	yewenbupt	00	0:00:01	11,320 K	11,336 K	5	91	10	2,209	815,686	2,060	
wmiprvse.exe	2776	NETWOR...	00	0:00:28	7,588 K	8,388 K	5	190	52	33,607	769,921	3,935	
POWERPNT.EXE	2736	yewenbupt	00	0:01:35	101,788 K	138,236 K	21	600	842	157,785	24,085,215	2,519,695	
conime.exe	2520	yewenbupt	00	0:00:00	3,112 K	3,112 K	1	0	0	557	0	0	
ApntEx.exe	2384	yewenbupt	00	0:00:00	3,224 K	3,340 K	2	0	0	1,471	0	0	
ApMsgFwd.exe	2244	yewenbupt	00	0:00:05	2,264 K	2,264 K	3	0	0	486	0	0	
AcSvc.exe	2216	SYSTEM	00	0:00:41	19,500 K	19,592 K	22	620	506	65,924	588,168	7,484	

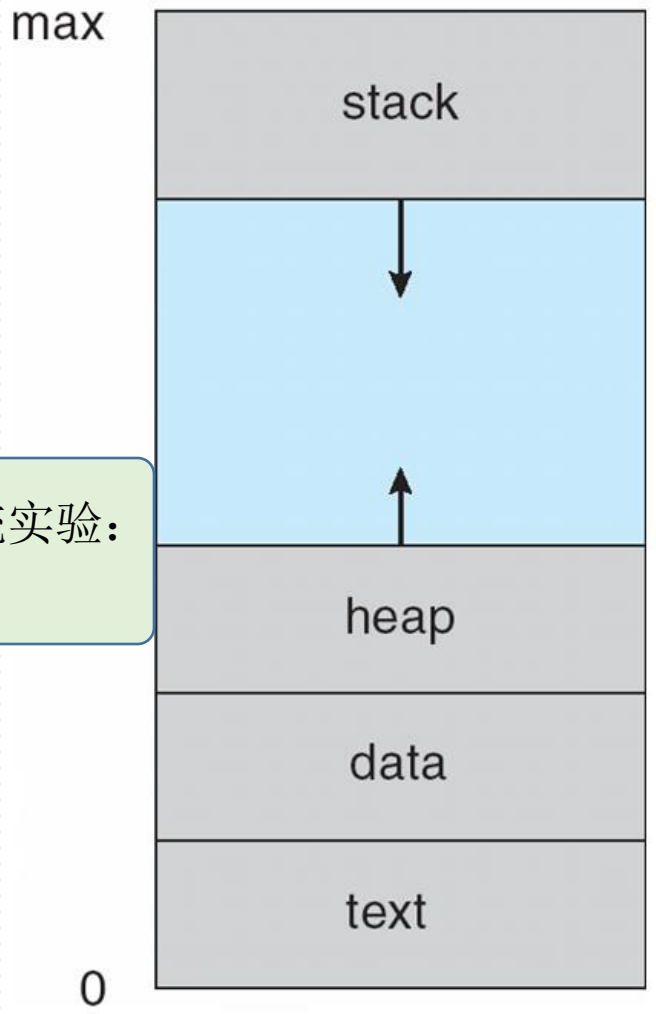
Process Concept

- A process consists of
  - the program code, also called **text section/文本段**
  - current activity including **program counter(PC,程序计数器)**, processor registers
    - indicating process execution traces, known as process context/进程上下文, at the instance of CPU switch



  - **stack** containing temporary data
    - function parameters, return addresses, local variables
  - **data section/数据段** containing global variables
  - **heap** containing memory dynamically allocated during run time
    - e.g. by malloc()

Linux操作系统实验：观察进程组成



Process Concept

- Program vs process
  - program is ***passive*** entity stored on disk (**executable file**), process is ***active***
    - program becomes process when executable file loaded into memory
  - execution of program started via GUI mouse clicks, command line entry of its name, etc
  - one program can be several processes
    - consider multiple users executing the same program

Process State

- A process is dynamic, and has its *lifetime.*
- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting(blocked)**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution

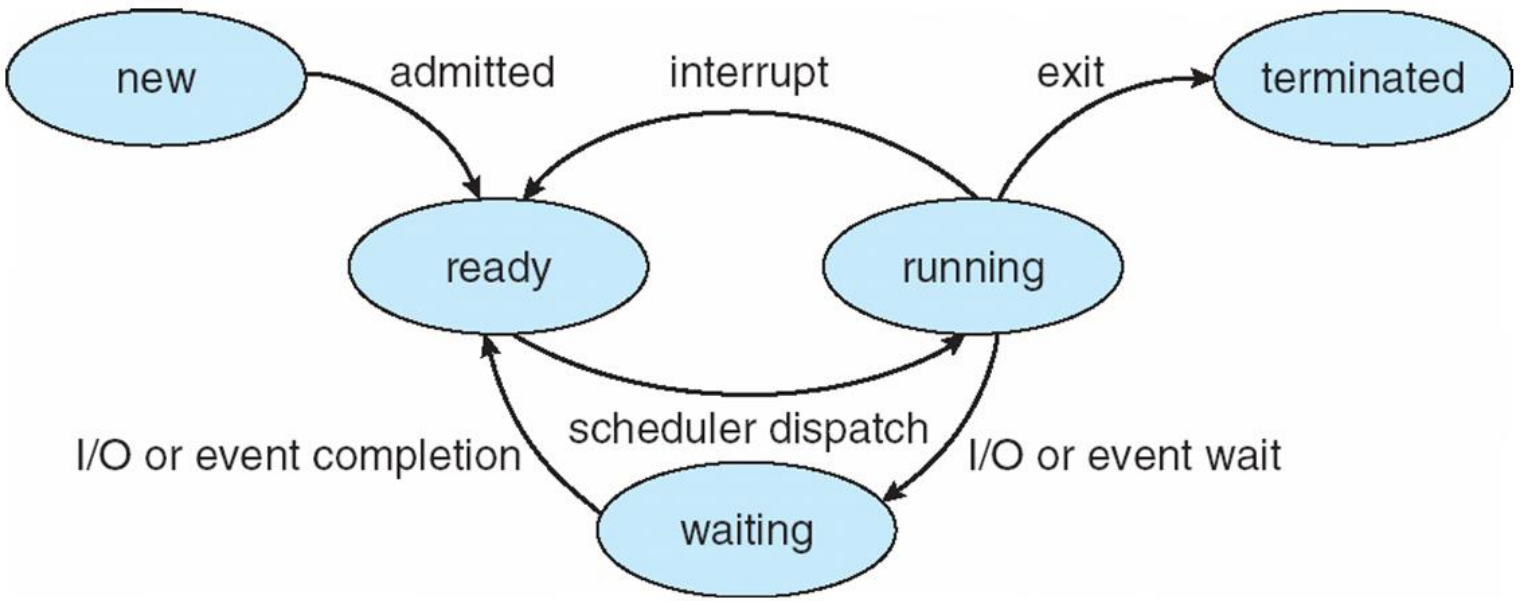
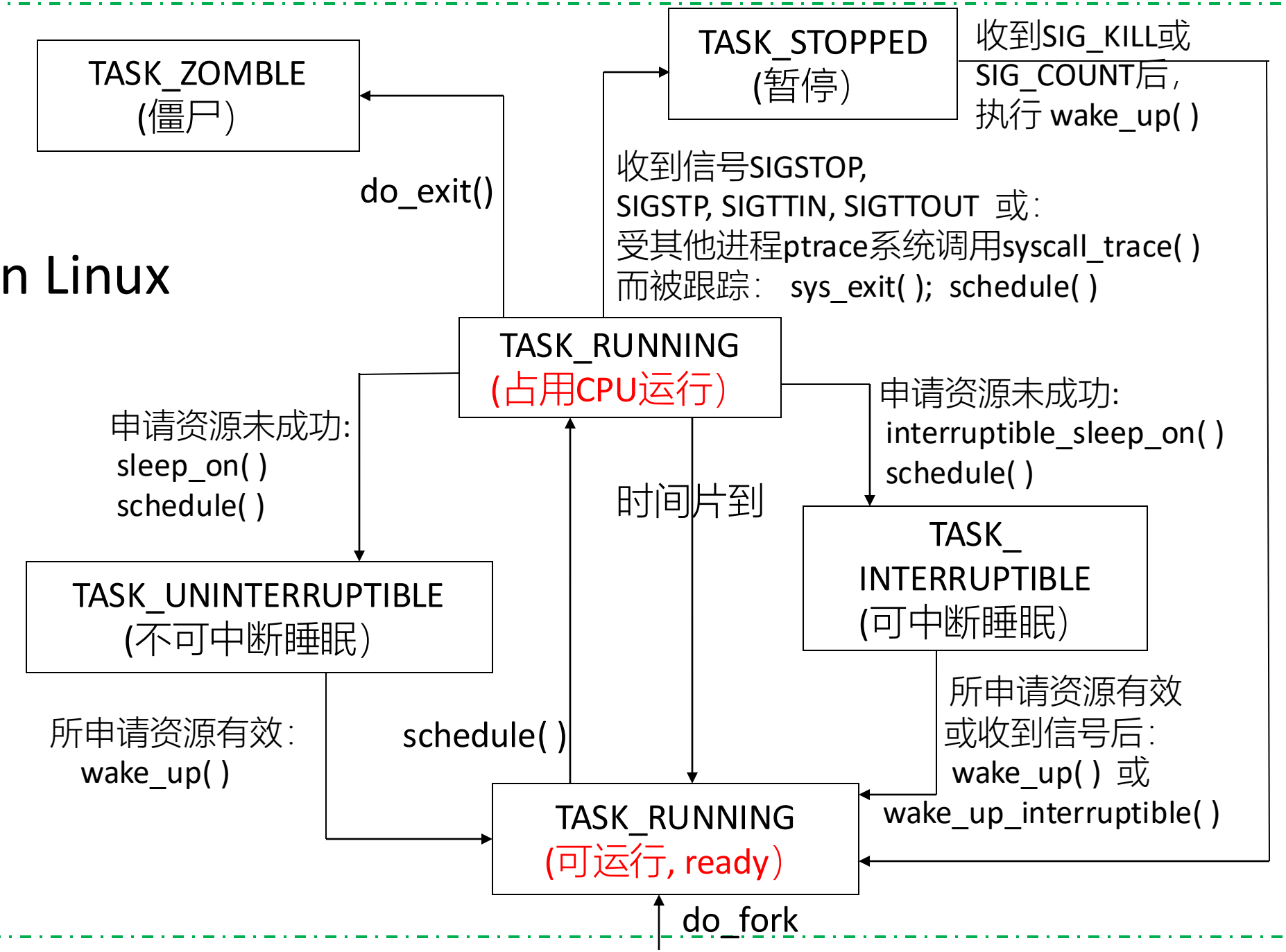


Fig. 3.2 Process States

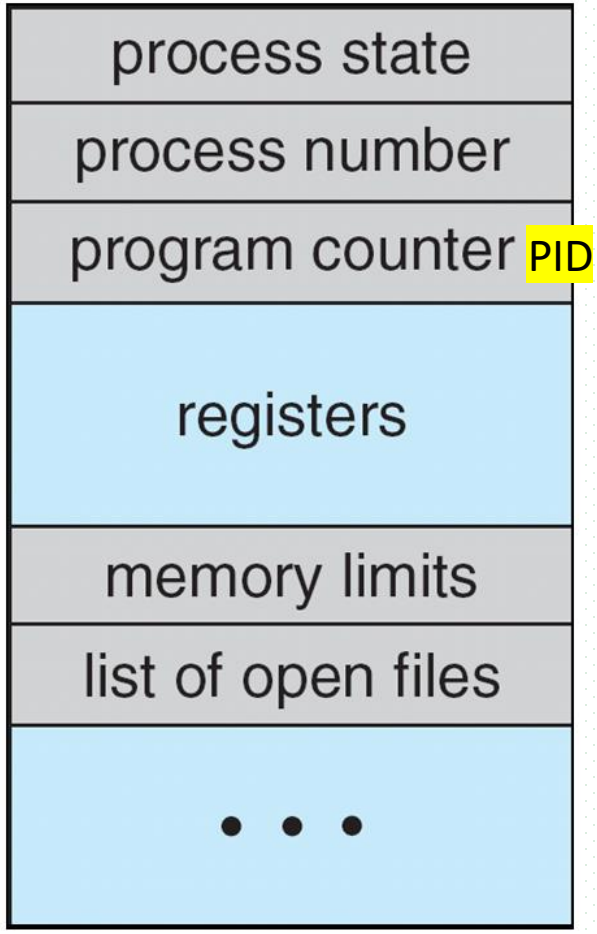
A practical OS such as Linux, may have different states from those in Fig.3.2

States in Linux



Process Control Block (PCB)

- PCB
  - a data structure in kernel used for OS to manage process
  - also called **task control block,** task structure task_struct in Linux
- Information associated with each process in PCB
  - 1 process state – ready, running, waiting, etc.
  - 2 program counter – location of instruction to next execute
  - 3 CPU registers – contents of all process-centric registers
  - 4 CPU scheduling information- priorities, scheduling queue pointers
  - 5 memory-management information – memory allocated to the process
  - 6 accounting information – CPU used, clock time elapsed since start time limits
  - 7 I/O status information – I/O devices allocated to process, list of open files



Process/task in Linux

- 在Linux中，进程process称为任务task，PCB对应任务结构task_struc
- 进程/task的虚拟地址空间分为用户虚拟地址空间和内核虚拟地址空间，所有进程共享内核虚拟地址空间，每个进程有各自独立的用户虚拟地址空间
- 线程
  - 内核(级)线程(Kernel-Level Thread, KLT)，无用户虚拟地址空间，仅在内核运行
  - 用户(级)线程(User-Level Thread, ULT)，共享用户进程的虚拟地址空间
- 线程组
  - 共享同一个用户虚拟地址空间的所有用户线程
- C 标准库进程术语和 Linux 内核进程术语对应关系

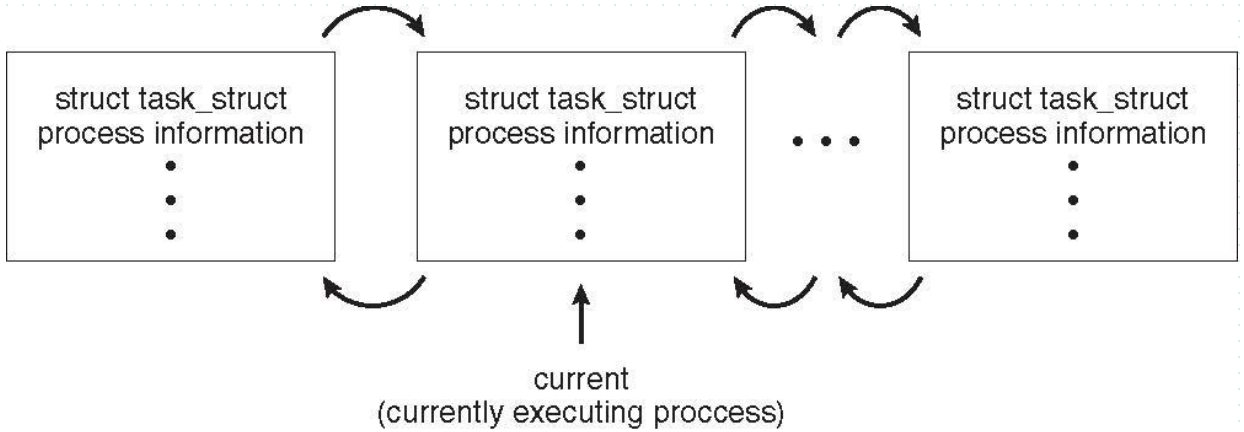
C 标准库进程术语	Linux 内核进程术语
包含多个线程的进程	线程组
只有一个线程的进程	进程或任务
线程	共享用户虚拟地址空间的进程

进程描述符Task_struc in Linux

- Represented by the C structure task_struct , i.e. PCB (part)

```
pid t_pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;   /* this process's parent */
struct list_head children;    /* this process's children */
struct files_struct *files;   /* list of open files */
struct mm_struct *mm;         /* address space of this process */
```

……



task_struct 主要成员

位于include/linux/sched.h

- task_struct进程描述符
- **_state**：指向进程状态
- ***stack**：指向内核栈
- pid：指向全局的进程号

  tgid：指向全局的线程组的标识符
- *real_parent：指向真实的父进程（创建它的进程）

  *parent：指向当前的父进程（可能因被跟踪、收养等而变化）
- 进程调度策略及其优先级
  - prio, static_prio, normal_prio, rt_priorit
- nr_cpus_allowed：
  - 允许进程在哪些处理器上执行
- ***mm**
  - 指向内存描述符，内核线程此项为空
- *active_mm
  - 指向内存描述符，内核线程运行时从进程借用
- ***fs**：文件系统信息

```
struct task_struct {//进程描述符 【部分】
#ifdef CONFIG_THREAD_INFO_IN_TASK
        /*
         * For reasons of header soup (see current_thread_info()), this
         * must be the first element of task_struct.
         */
        struct thread_info              thread_info;
#endif
        unsigned int                    _state; //指向进程状态
#ifdef CONFIG_PREEMPT_RT
        /* saved state for "spinlock sleepers" */
        unsigned int                    saved_state;
#endif
        /*
         * This begins the randomizable portion of task_struct. Only
         * scheduling-critical items should be added above here.
         */
        randomized_struct_fields_start

        void                            *stack;       //指向内核栈
        refcount_t                      usage;
        /* Per task flags (PF_*), defined further below: */
        unsigned int                    flags;
        unsigned int                    ptrace;

        // ......
};
```

Process Context (进程上下文)

- A process' context describes/records its executing status when running on CPU, and consists of
  - values/contents of the CPU registers, including general-purpose registers and program counter(PC)/instruction pointer(IP) etc., when the process running on CPU,
    - e.g. in Intel x86-64 CPU

1. 通用寄存器: RAX, RBX, RCX, RDX
2. 栈寄存器: RSP(栈顶指针寄存器), RBP(栈基址寄存器)
3. 源变址和目标变址寄存器: RSI, RDI
4. 指令寄存器: RIP
5. 传参寄存器: RCX, RDX, R8, R9
6. Scratch寄存器: RBX,R12, R13, R14, R15(Scratch)

....

  - the process states, e.g. ready, running, waiting, ...
  - memory management information etc., e.g. address space the process located in memory
  - ...

CPU Switch From Process to Process

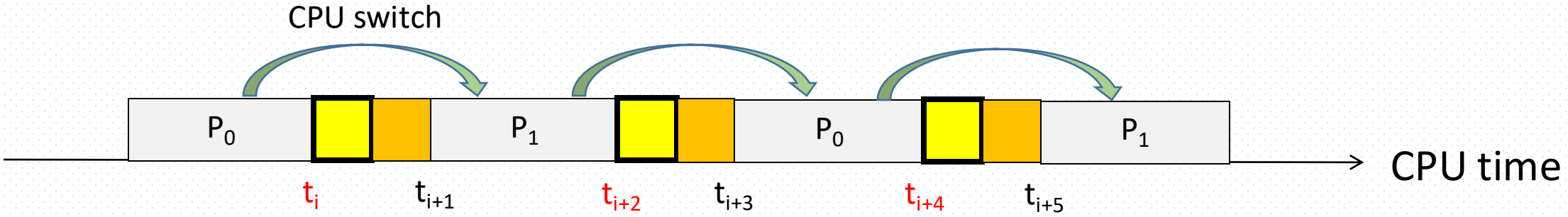
- In a system where multiple programs concurrently execute, several processes alternatively occupies and runs on the CPU, resulting in CPU switch/CPU切换 from process to process

- The context of a process at the moment of CPU switch are saved in its PCB
  - e.g. t_i, t_{i+2}, t_{i+4}, … , the moment when a process running in user mode is interrupted and gives up CPU
  - CPU switches from the user mode to kernel mode

P_0 and P_1 runs, in user mode

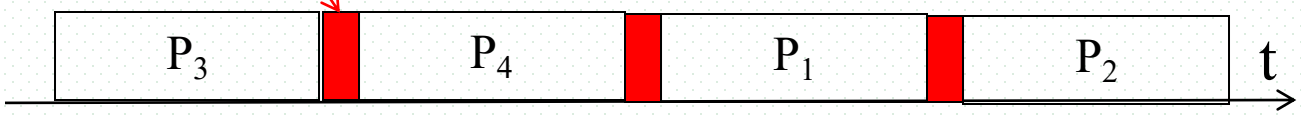
OS breaks off the execution of P_0 or P_1, and saves its context, in kernel mode

OS reloads the context of P_1 or P_0, and restarts its execution, in kernel mode



Process Context Switch/CPU Switch (进程上下文切换, CPU切换)

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
  - refer to ▷
- OS executes context-switch in kernel mode, and the switch time is overhead; the system does no useful work while switching
  - the more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once



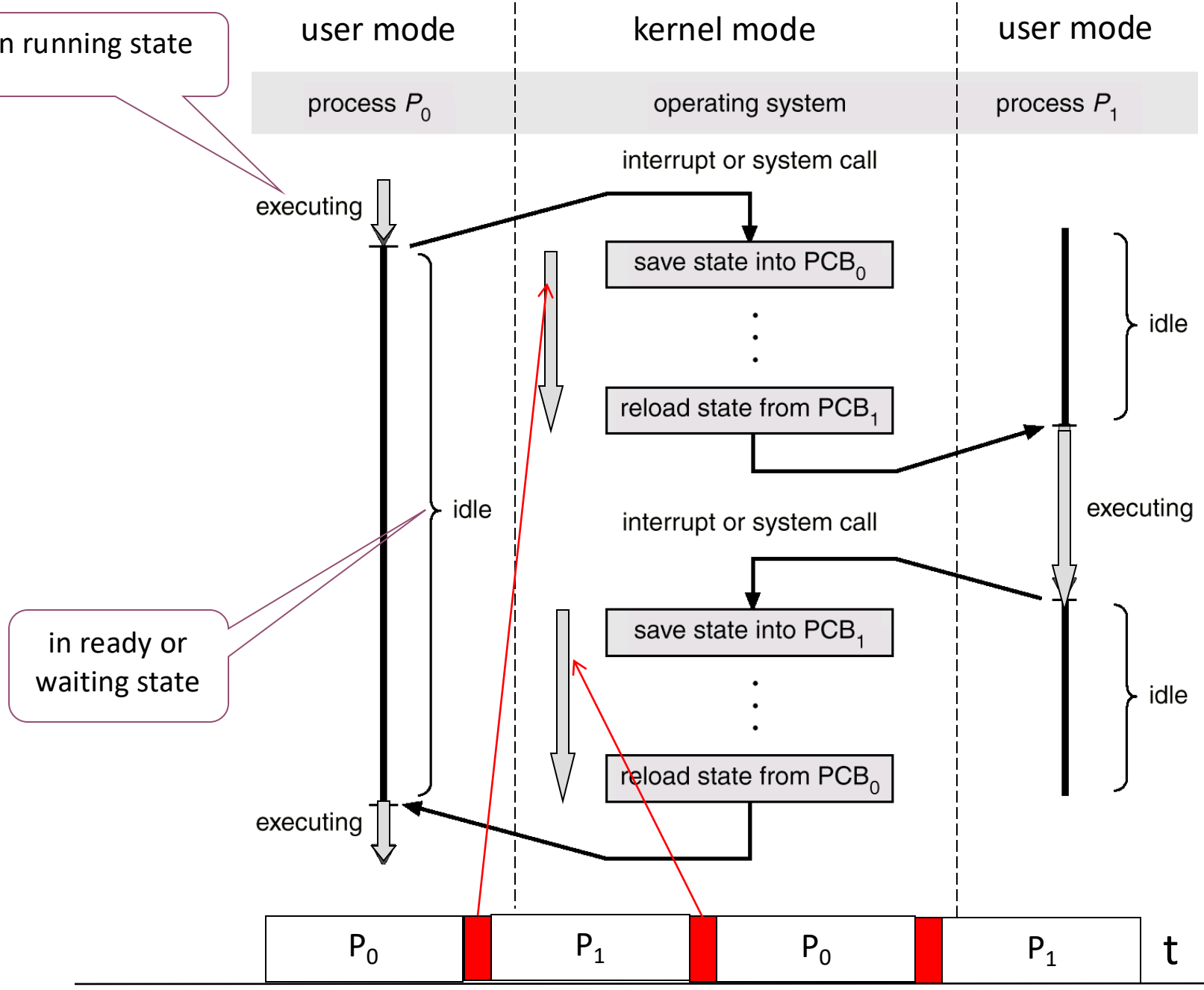


Fig. 3.4 **CPU switch** from process to process

（Intel x86 平台）进程上下文切换开销包括

直接开销

- 1. 切换页表全局目录
- 2. 切换内核态堆栈
- 3. 切换硬件上下文（进程恢复前，必须装入寄存器的数据统称为硬件上下文）
  - ip(instruction pointer)：指向当前执行指令的下一条指令
  - bp(base pointer): 用于存放执行中的函数对应的栈帧的栈底地址
  - sp(stack poinger): 用于存放执行中的函数对应的栈帧的栈顶地址
  - cr3:页目录基址寄存器，保存页目录表的物理地址
- 4. 刷新TLB
- 5. 系统调度器的代码执行

间接开销-降低CPU访存速度

- 切换到一个新进程后，由于缓存cache并不热，CPU速度运行会慢一些。如果进程始终都在一个CPU上调度还好一些，如果跨CPU的话，之前热起来的TLB、L1、L2、L3因为运行的进程已经变了，所以以局部性原理cache起来的代码、数据将失效，导致新进程穿透到内存的IO会变多

Threads

- So far, process has a single thread of execution

- Consider multi-thread programming in Chapter 4, having multiple program counters per process
  - multiple locations can execute at once 
    - multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB

- Refer to next chapter

3.2 Process Scheduling

- Selecting processes and allocating CPU, devices etc. resources to them
- Maximize CPU usage, quickly switch processes onto CPU for time sharing
- Also for other resources, e.g. I/O devices
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Process Migration/迁移 between The Various Queues

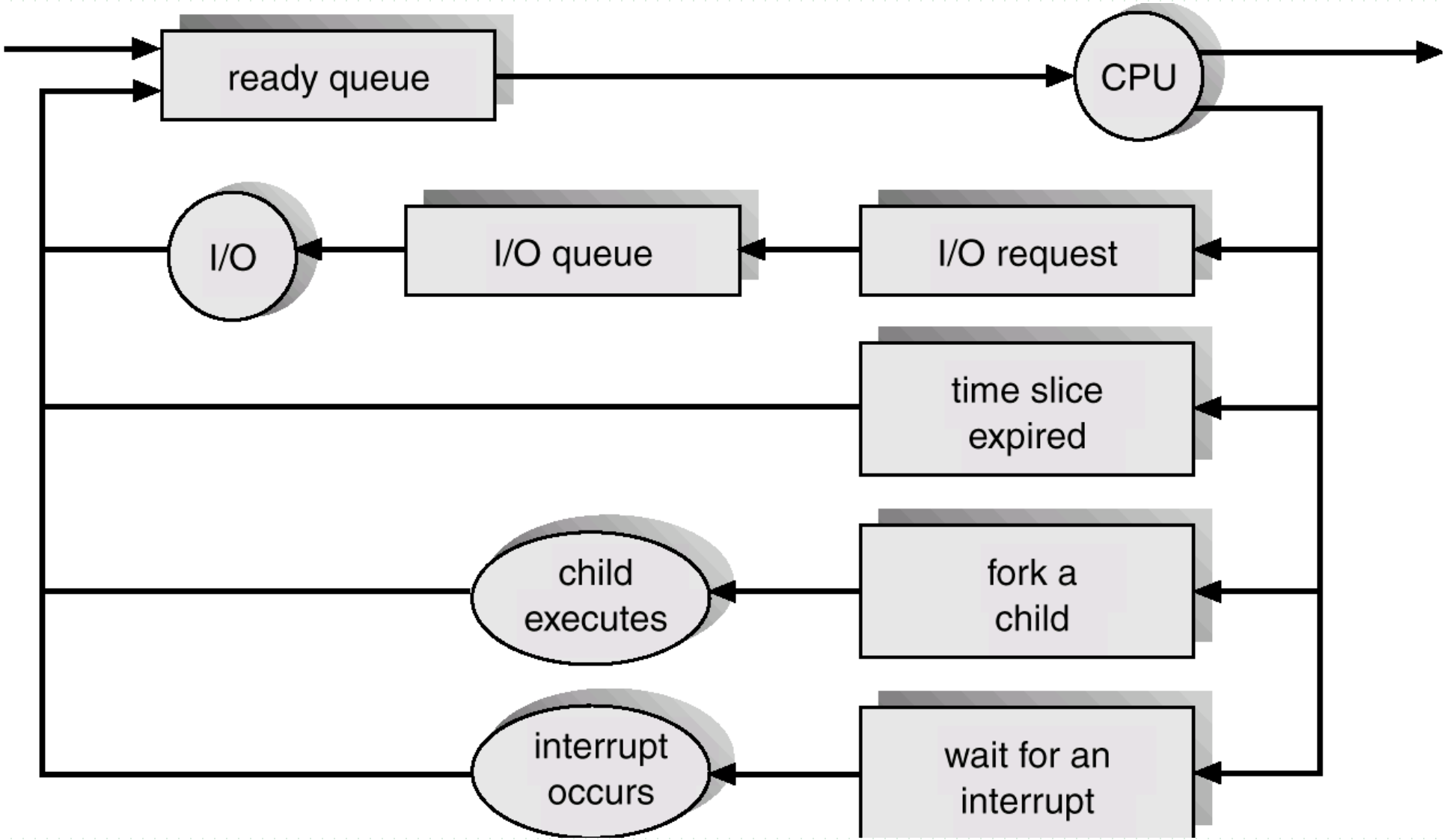


Fig. 3.7 Representation of Process Scheduling

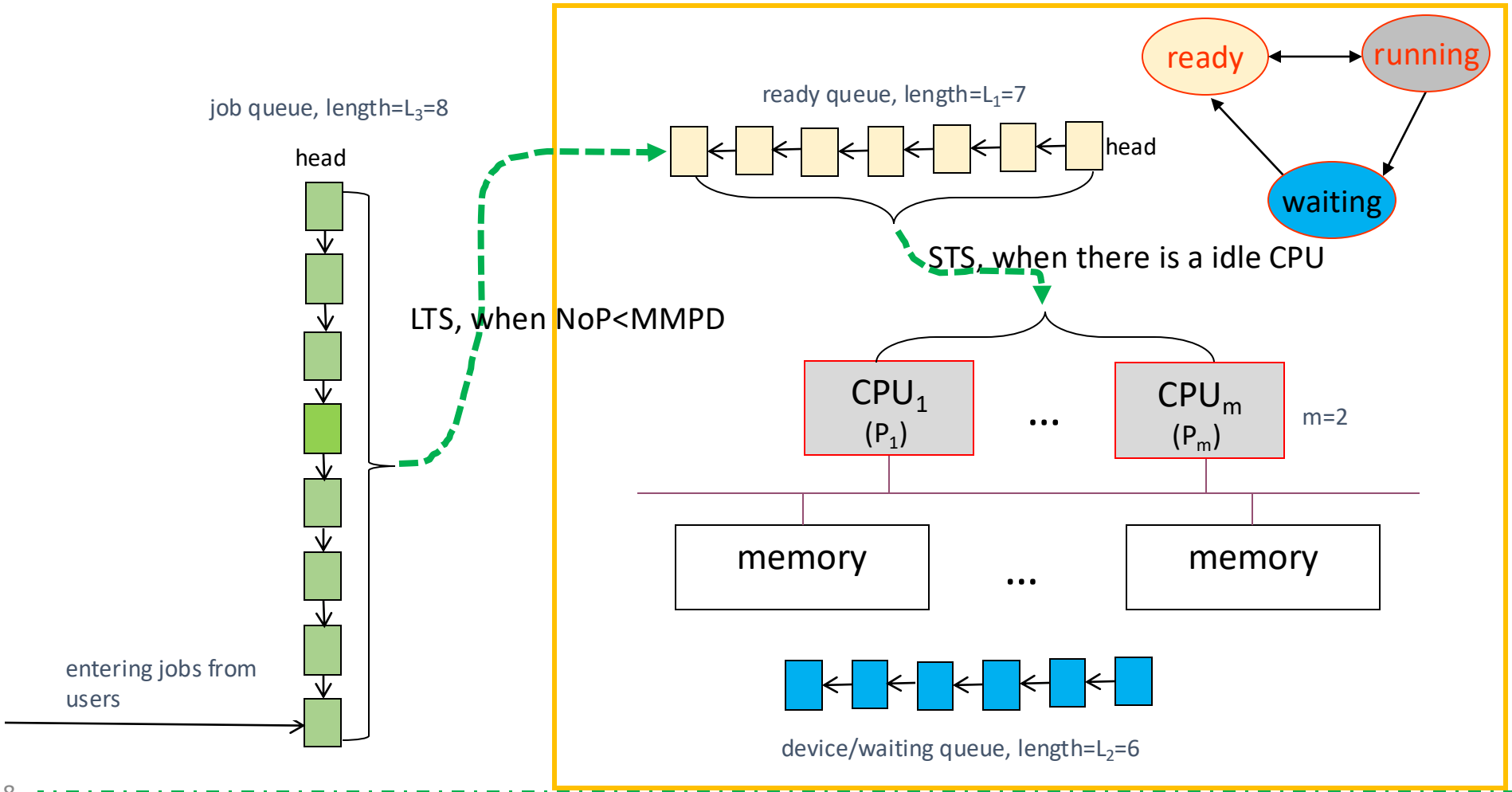
Schedulers

- Degree of multiprogramming (多道程序设计粒度)
  - the number of process in (main) memory
- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - sometimes the only scheduler in a system, e.g. in PC
  - short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- In batch systems (e.g. mainframe systems), more processes (or user jobs) are submitted to the system than can be executed immediately. These processes/jobs are spooled ***as jobs*** to a mass-storage device (typical a disk) , where they are kept for later execution
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**

Long-term scheduling(LTS) vs Short-term scheduling(STS)

要求：NoP ≤ MMPD

最大多道程序设计粒度MMPD，允许的并发进程最大数目	用户提交作业总数NoJ	后备态作业总数 L_3	进程总数NoP，多道程序设计粒度，运行态作业总数	ready态进程总数L_1	running态进程总数m	waiting态进程总数 L_2
16	23 = 8+15	8	15=7+2+6	7	2	6

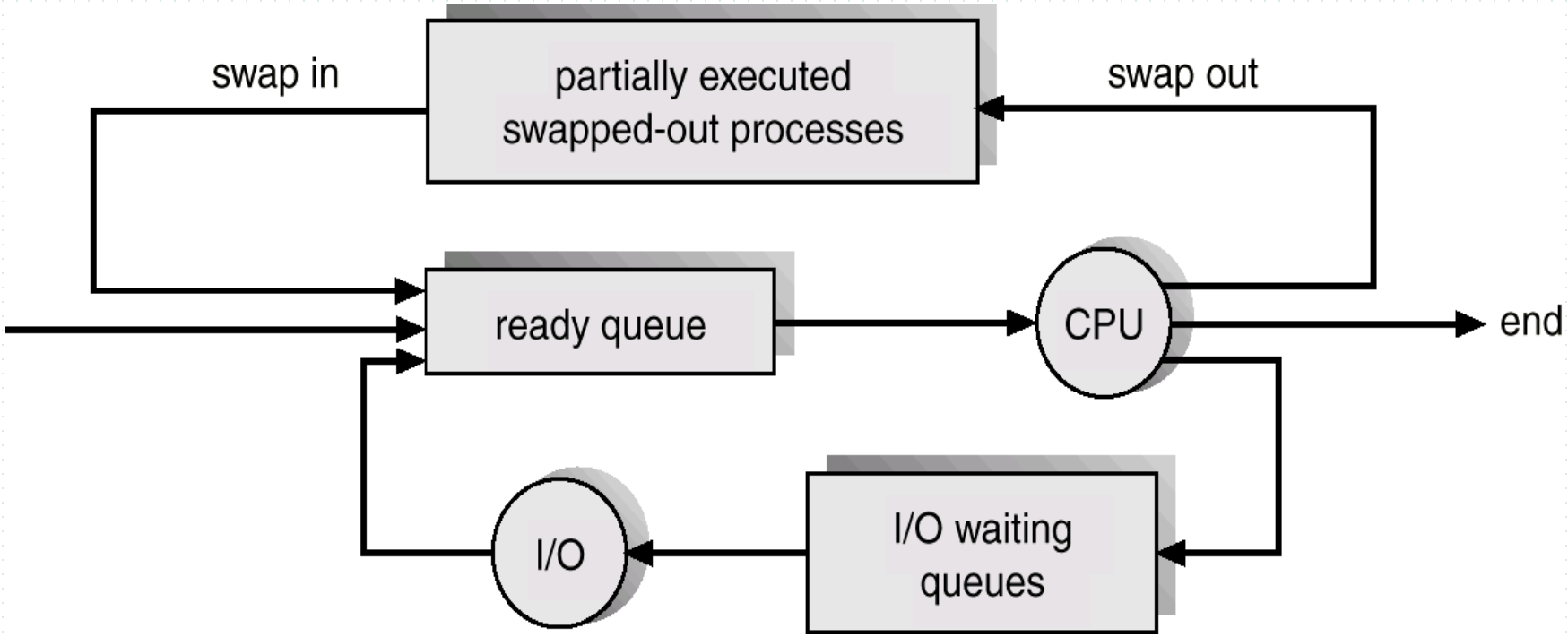


Schedulers

- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts, also called I/O-intensive process
    - I/O受限，I/O密集型
    - e.g. access data in database(DB) stored on disk
    - DPU, data processing unit
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts, also called computation-intensive process
    - CPU受限，CPU密集型
    - e.g. scientific computing, high-performance computing (HPC) on SuperComputer
- Long-term scheduler strives for good process mix

Addition of Medium Term Scheduling

- In some operating systems, such as time-sharing systems, medium term scheduling exists
  - to control the degree of multiprogramming, guaranteeing the resources (such as main memory) demanded by all processes in memory has not overcommitted available resources
- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - remove process from memory, store on disk, bring back in from disk to continue execution: **swapping/交换**

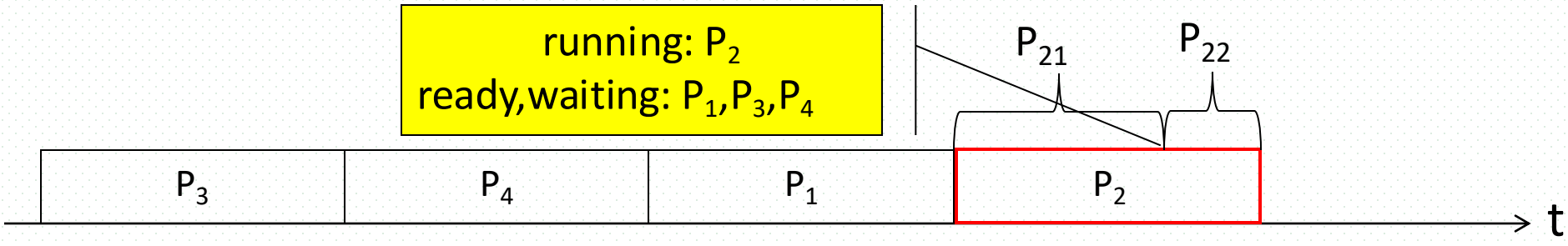


Example

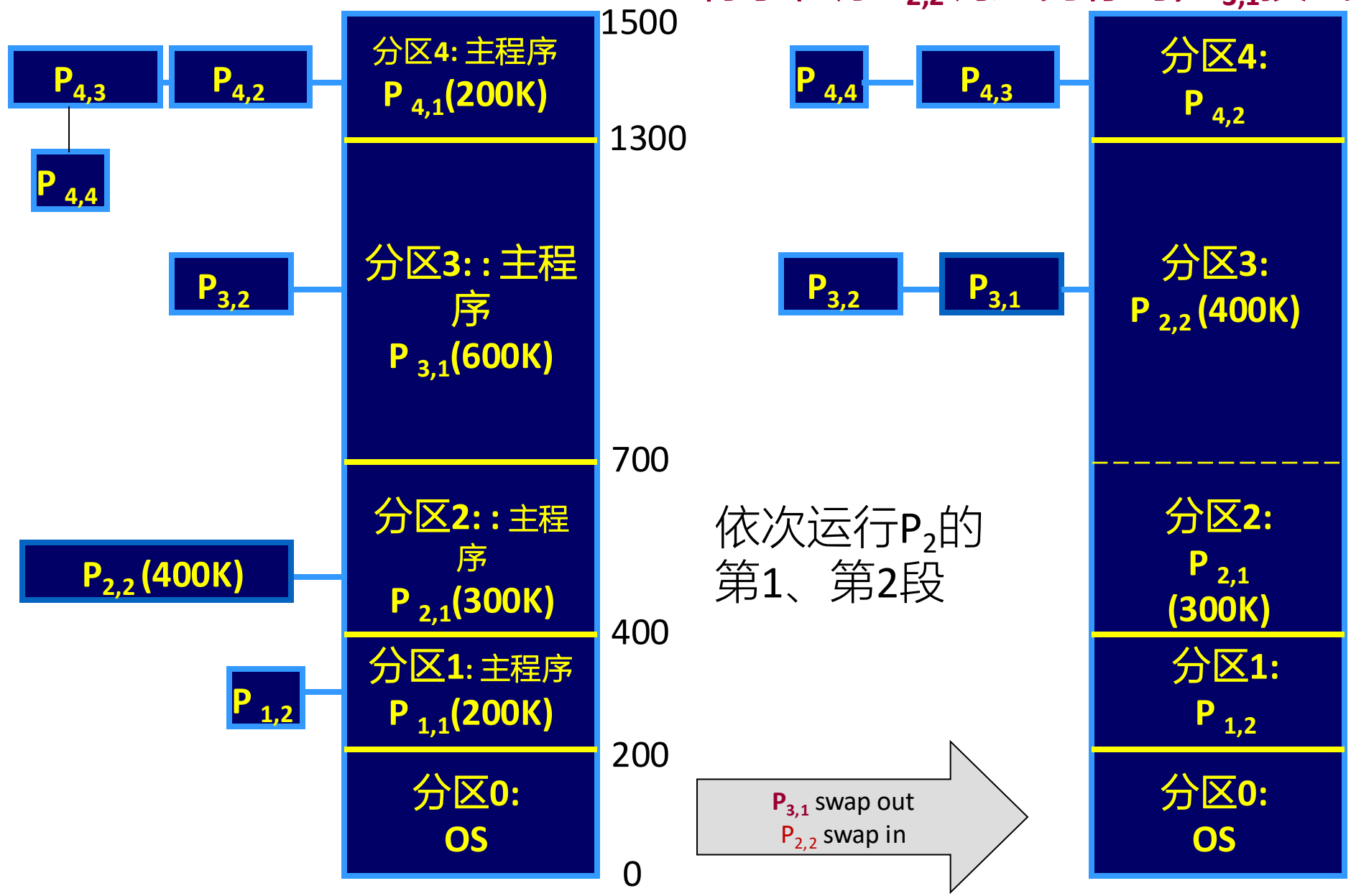
- Four processes P_1, P_2, P_3 and P_4 concurrently run in main memory
  - P_1, P_2, P_3 and P_4 are 300K, 700K, 800K, and 700K total in size respectively, 300+700+800+700=2500
  - main memory is divided into five sections, i.e, **section0, section1, section2, section3**, and **section4**, allocated to **OS** and the four **processes** to be loaded to execute
  - the size of each section is 200K, 200K, 300K, 600K, 200K respectively, and is smaller than the size of the process running in it
    - 200 + 200 + 300 + 600 + 200 = 1500 < 2500

Example

- each process is divided into several parts, and each part is loaded into memory to run *sequentially*
  - P_1(300K) = {main program P_{11}(200K) , subroutine P_{12}(100K)},
  - P_2 (700K) ={main program P_{21} (300K), subroutine P_{22} (400K) }
  - P_3 (800K) ={main program P_{31} (600K), function P_{32} (200K) }
  - P_4 (700K) ={main program P_{41} (200K), procedure P_{42} (200K), P_{43} (200K), P_{44} (100K)}
- initially, each process executes its first part e.g. main program, in the section allocated to it, as described in Fig.4.0.2
- when the subroutine $P_{2,2}$ (400K) are loaded to memory, the main program $P_{3,1}$ (600K) , which is still in execution, is swapped out

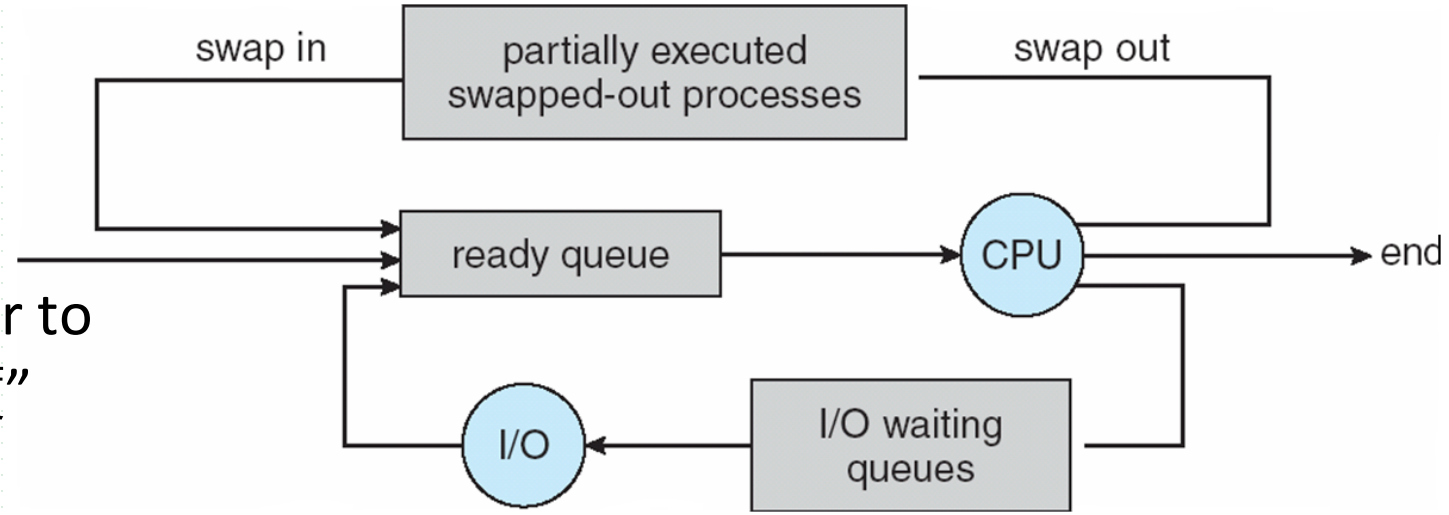


time-sharing scheduling


各个进程运行其第一段主程序
将子程序P2,2调入内存时, P3,1换出
P4,3
P4,2
P4,4
P3,2
P2,2 (400K)
P1,2
分区4: 主程序
P 4,1(200K)
分区3: : 主程序
P 3,1(600K)
分区2: : 主程序
P 2,1(300K)
分区1: 主程序
P 1,1(200K)
分区0:
OS
1500
1300
700
400
200
0
依次运行P2的
第1、第2段
P3,1 swap out
P2,2 swap in
P 4,4
P4,3
P3,2
P3,1
分区4:
P 4,2
分区3:
P 2,2 (400K)
分区2:
P 2,1
(300K)
分区1:
P 1,2
分区0:
OS

Addition of Medium Term Scheduling

- Medium term scheduling is responsible for swapping (交换/倒换)
  - swapping out
    - to reduce degree of multiprogramming and free memory or resources, removing processes in ready or running states from main memory and into *swapping section* on disks
    - the process that is swapped out is delayed to execute
  - swapping in
    - reintroducing the processes on the disk into memory into memory, restarting their executing



- For more details about scheduling, refer to "Appendix 3-1 调度的层次与作业调度"

Multitasking in Mobile Systems

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - single **foreground** process- controlled via user interface
  - multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use

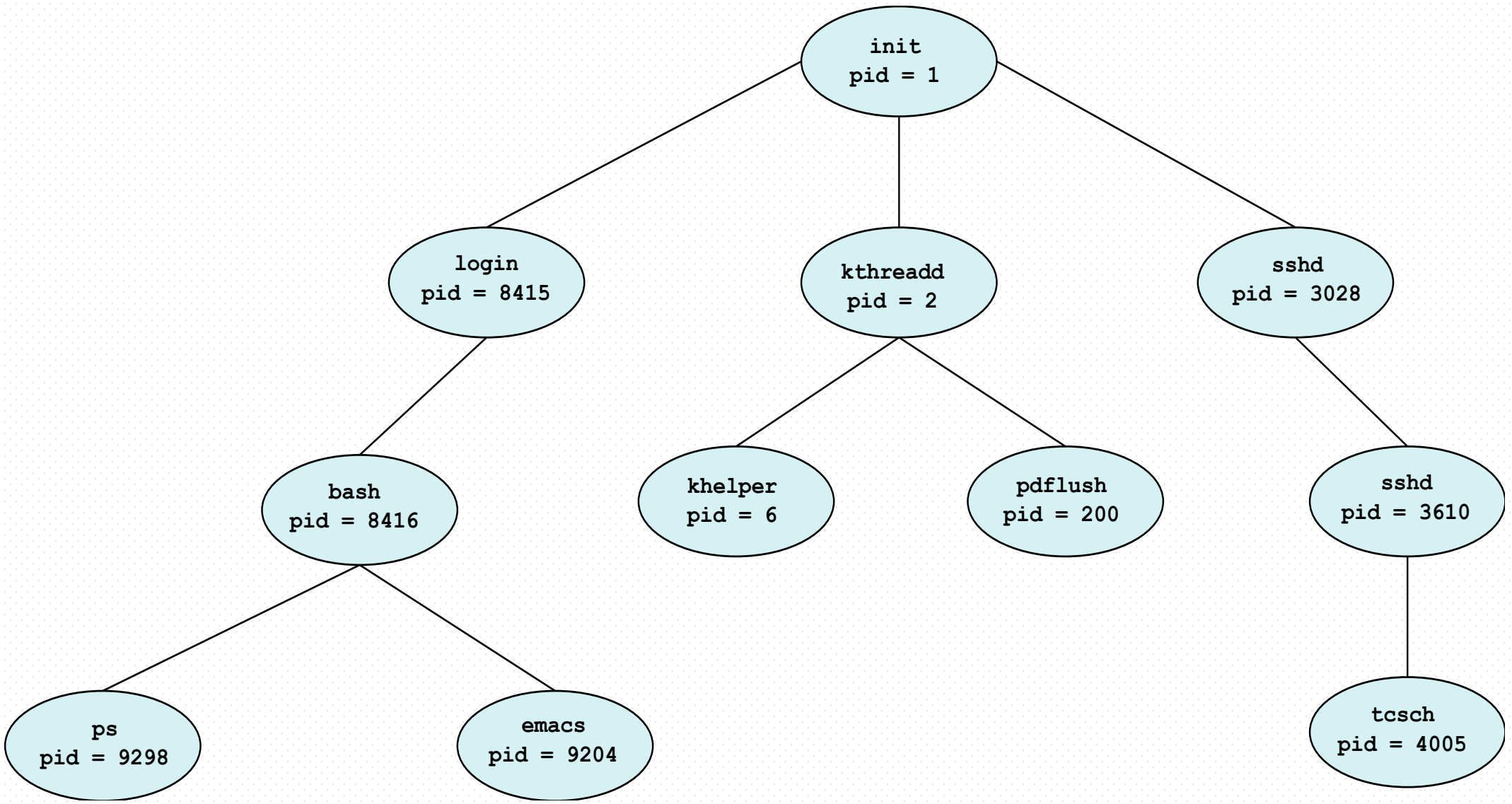
3.3 Operations on Processes

- OS must provide mechanisms, by system calls, for:
  - process creation
  - process termination
  - and so on as detailed in following chapters

Process Creation

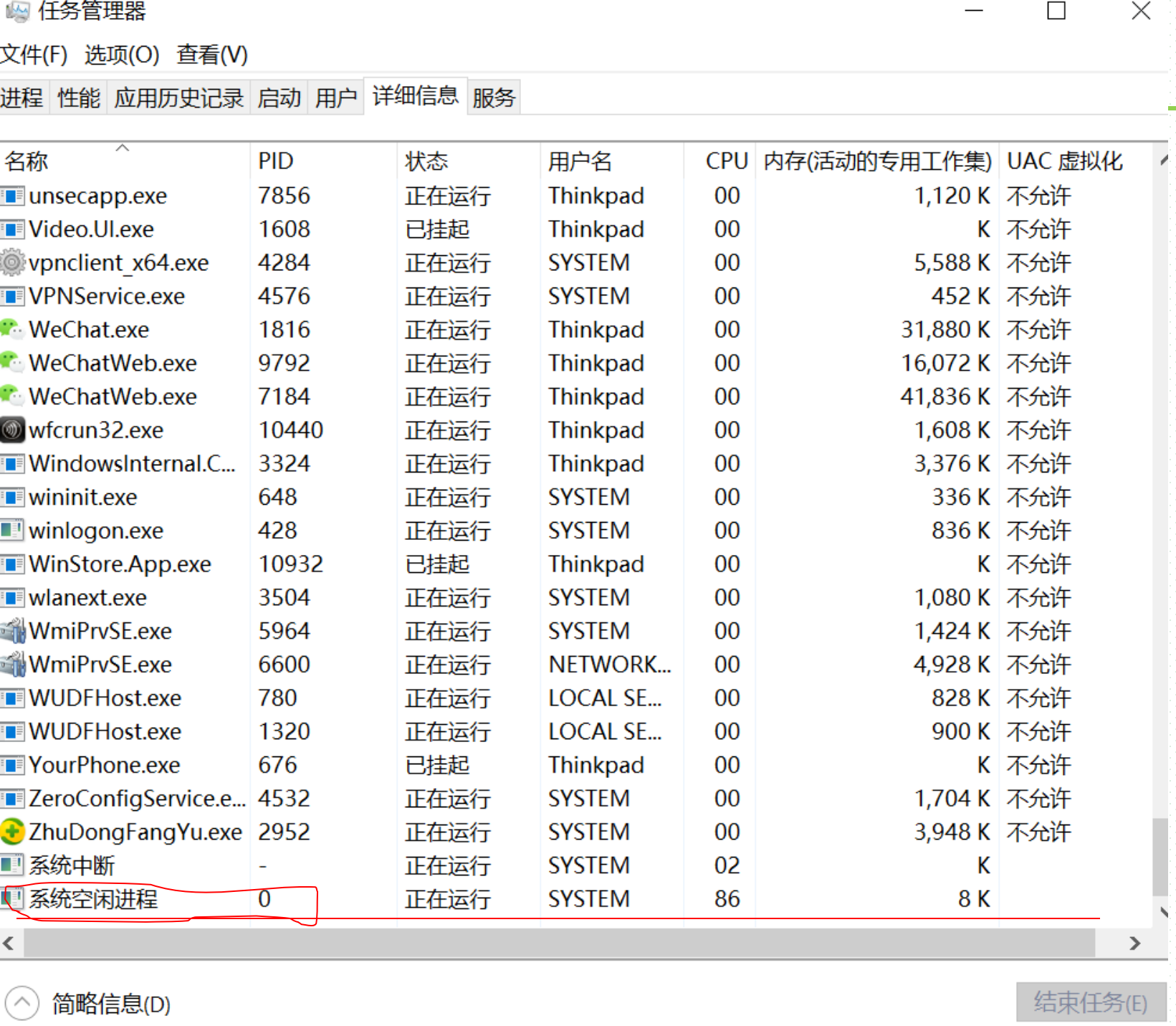
- OS and other processes use ***creation*** primitive /system call to create a new process
- Actions taken by OS when creating a process include
  - load the program code into main memory allocated to this process
  - allocate **resources** (memory, I/O devices, files) to the process
  - **build the PCB for this process**
  - insert the PCB into ready queue, (and the process change into **ready** state)
- The process being created is in ***new*** state
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes

A Tree of Processes in Linux



Process Creation

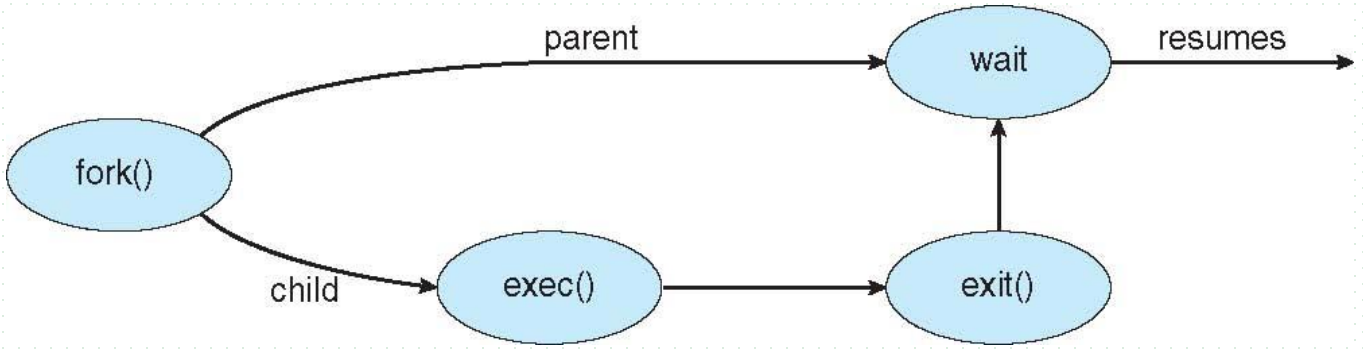
- Resource sharing options
  - parent and children share all resources
  - children share subset of parent′s resources
  - parent and child share no resources
- Execution options
  - parent and children execute concurrently
  - parent waits until children terminate



- 内核初始化init模块
  - 加载启动其它内核模块，自身变为0号idle进程
  - 为系统内其他进程的祖先节点

Process Creation

- Address space
  - total range of memory locations addressable by the process
- Two modes for Address space of child process
  - child duplicate of parent, so parent and child processes can communicate easily, for example, by memory-sharing scheme
    - E.g. Linux/Unix
  - Child has a program loaded into it, i.e., the child has its independent address space
- UNIX/Linux examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process′ memory space with a new program



C Program Forking Separate Process

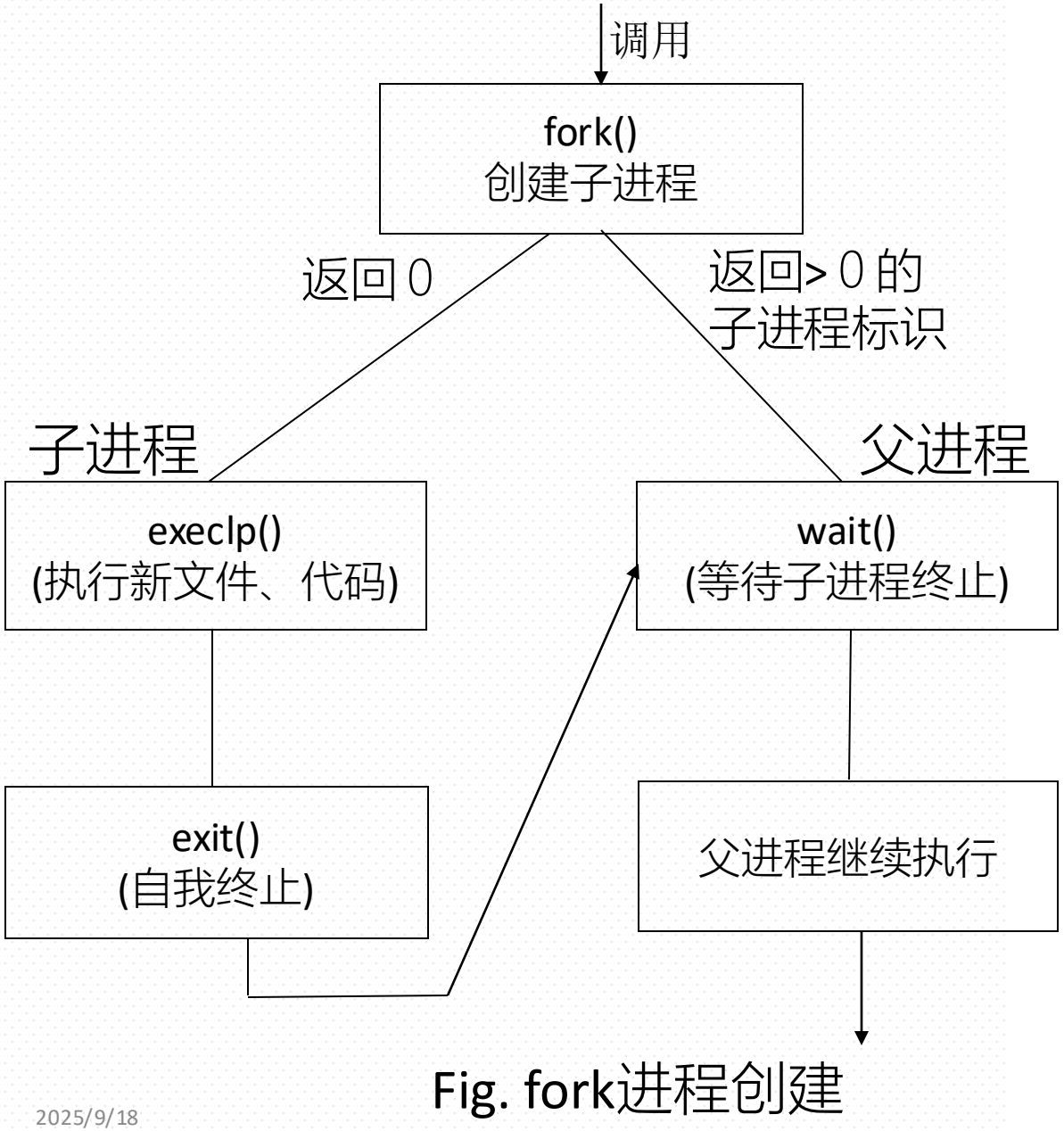


Fig. fork进程创建

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Linux操作系统实验：进程创建

Process creation and exec()

- E.g. 键盘输入程序(fork, exec, wait系统调用)

```
while {
    显示命令提示符;
    等待用户命令键入命令;
    接收并分析命令行;
    if  (pid=fork())>0
        if (无&）wait(pid);
    else
        exec( 程序名，参数)
调用；返回；赋值；判断
```

父进程

fork()成功，返回子进程pid>0

```
while {
    显示命令提示符;
    等待用户命令键入命令;
    接收并分析命令行;
    if  (pid=fork())>0
        if (无&）wait(pid);
    else
        exec( 程序名，参数)
        返回；赋值；判断
```

fork()创建的子进程

Creating a Separate Process via Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

   /* allocate memory */
   ZeroMemory(&si, sizeof(si));
   si.cb = sizeof(si);
   ZeroMemory(&pi, sizeof(pi));

   /* create child process */
   if (!CreateProcess(NULL, /* use command line */
    "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
    NULL, /* don't inherit process handle */
    NULL, /* don't inherit thread handle */
    FALSE, /* disable handle inheritance */
    0, /* no creation flags */
    NULL, /* use parent's environment block */
    NULL, /* use parent's existing directory */
    &si,
    &pi))
   {
     fprintf(stderr, "Create Process Failed");
     return -1;
   }
   /* parent will wait for the child to complete */
   WaitForSingleObject(pi.hProcess, INFINITE);
   printf("Child Complete");

   /* close handles */
   CloseHandle(pi.hProcess);
   CloseHandle(pi.hThread);
}
```

Process Startup in Linux

- Step1. 载入(loading)
  - OS加载可执行文件：将进程可执行文件从磁盘读取到内存中，并且在该进程的虚拟地址空间中分配内存空间
    - 可执行文件：代码，数据
  - 采用虚拟内存VM技术，只加载进程的一部分可执行文件
- step2. 解析 ELF 文件头
  - Linux 系统中，可执行文件采用 ELF（Executable and Linkable Format）格式
  - OS通过解析文件头，了解进程程序的入口点、可执行段和数据段的位置等信息
- Step3.分配栈空间
  - OS为进程分配进程栈，用于存储函数调用的的参数和返回地址、局部变量等数据
- Step4. 初始化寄存器（进程上下文context）
  - OS初始化task_struc中的进程context寄存器，包括程序计数器（PC）、堆栈指针（SP）和其它通用寄存器

Process Startup in Linux（续）

- Step5. 在 Intel 处理器上，设置保护模式
  - 进程启动时进入保护模式
  - 在保护模式下，进程的内存空间被划分为若干个段，每个段有自己的访问权限
  - 进程可以使用段寻址访问内存
- Step6. 设置代码段和数据段
  - 在x86-64位架构下，为禁止使用 Intel 处理器的段寻址功能，将代码段和数据段的段号设置为 0
  - 进程只使用基于偏移量的地址寻址方式
- Step7. 转移CPU控制权，启动进程执行
  - OS将CPU控制权转移给进程代码入口点，即根据进程代码入口点设置程序计数器，、设置进程上下文中的各个寄存器值
  - CPU转入用户模式，开始执行进程代码

Step1~Step6在进程的创建/new状态下完成，之后进程进入ready状态；
一旦进程被OS调度器分配了CPU，通过Step7进入running态

Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call in Unix/Linux.
  - returns status data from child to parent (via **`wait()`**)
  - process′ resources are deallocated by operating system

- Parent may terminate the execution of children processes using the **abort()/夭折**system call. Some reasons for doing so:
  - child has exceeded allocated resources
  - task assigned to child is no longer required
  - the parent is exiting and the operating systems does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading/级联 termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call . The call returns status information and the pid of the terminated process, enabling the parent to know which child is terminated

```
pid = wait(&status)
```

- **Zombie process**（僵尸）
  - child is terminated by exit() and its resource is released, but no parent invoke wait, child is at the Zombie state
- **Orphan process**（孤儿）
  - If parent terminated without invoking **`wait`**, the child becomes a orphan
  - init is then taken as its parent

Multiprocess Architecture – Chrome Browser【略】

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer/渲染** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox/沙箱** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in

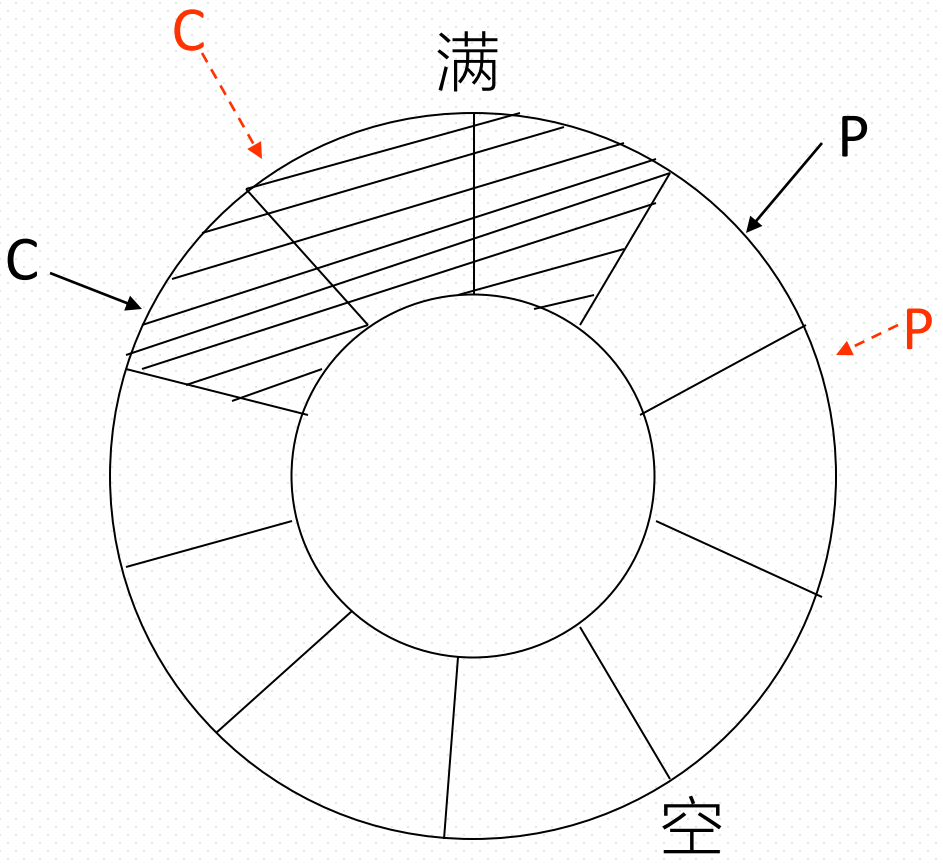
3.4 Interprocess Communication/进程间通信

- Processes within a system may be **independent** or **cooperating/协作**
- **Independent** process cannot affect or be affected by the execution of another process
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Concurrent executing of cooperating processes requires OS to provide mechanisms allowing processes
  - to **synchronize** their actions (more details in Chapter 6)
  - to **communicate** with one another, i.e. InterProcess Communication (IPC) (§3.4)

Process Synchronization: Example1- Producer-Consumer Problem

- Paradigm for cooperating processes, in which *producer* process produces information that is consumed by a *consumer* process, through a sharing **buffer**

1. 容量为N的环形缓冲区
   满缓冲区头指针C
   空缓冲区头指针P
2. *n*个生产者，*m*个消费者.
   多个生产者和消费者并发运行
3. 生产者—产生数据；
   写入指针P指
   向的空缓冲区；
   指针P前移。
4. 消费者—从指针C指向的满
   缓冲区中取数据；
   指针C前移



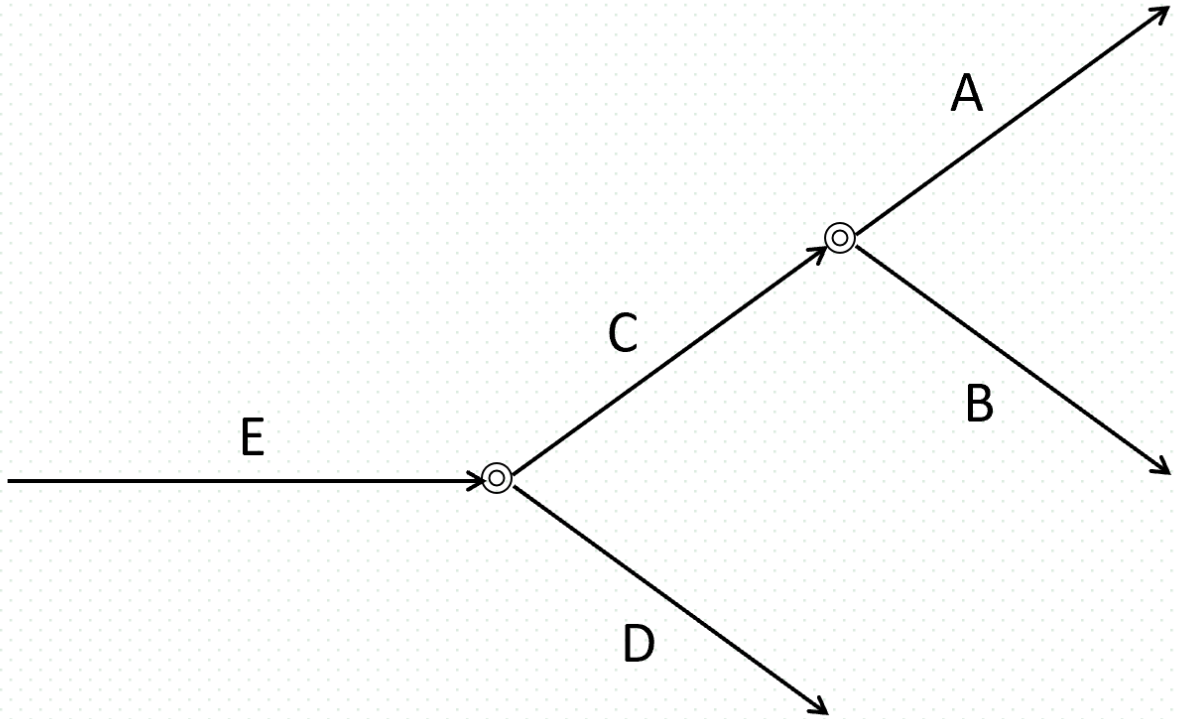
Producer-Consumer Problem

- Paradigm for cooperating processes, in which *producer* process produces information that is consumed by a *consumer* process, through a sharing **buffer**
- Cooperating
  - 生产者写数据时须有空缓冲块。如果不满足，则阻塞
  - 2个生产者不能同时向同一 空缓冲块写数据
  - 消费者取数据时，须有满缓冲块。如果不满足，则阻塞
  - 2个消费者不能同时从同一 满缓冲块取数据
  - 生产者和消费者不能同时对同一缓冲块进行读写操作

- 许多实际问题可抽象为生产者-消费者问题/模型， 如基于邮箱的进程通信, 共享内存通信方式

Process Synchronization: Example2

- 现有5个并发进程A、B、C、D、E，进程C完成之后A和B才开始执行，进程E完成之后C和D才开始执行
- 使用信号量和wait()、signal操作，描述上述进程间的同步关系
- 要求
  - 说明信号量含义及其初值，信号量名称应有明确含义，与题意相符
- 答案
  - 5个进程间同步关系

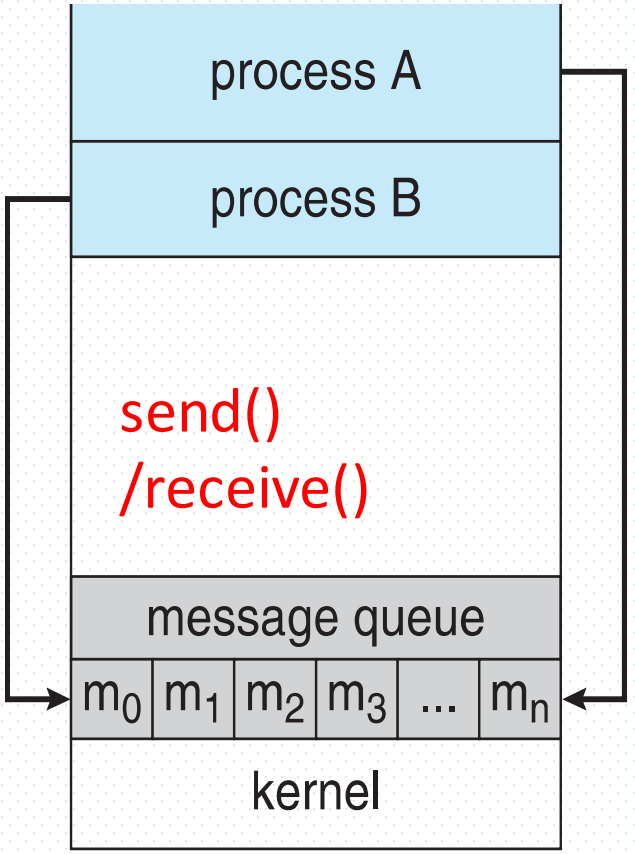


Interprocess Communication

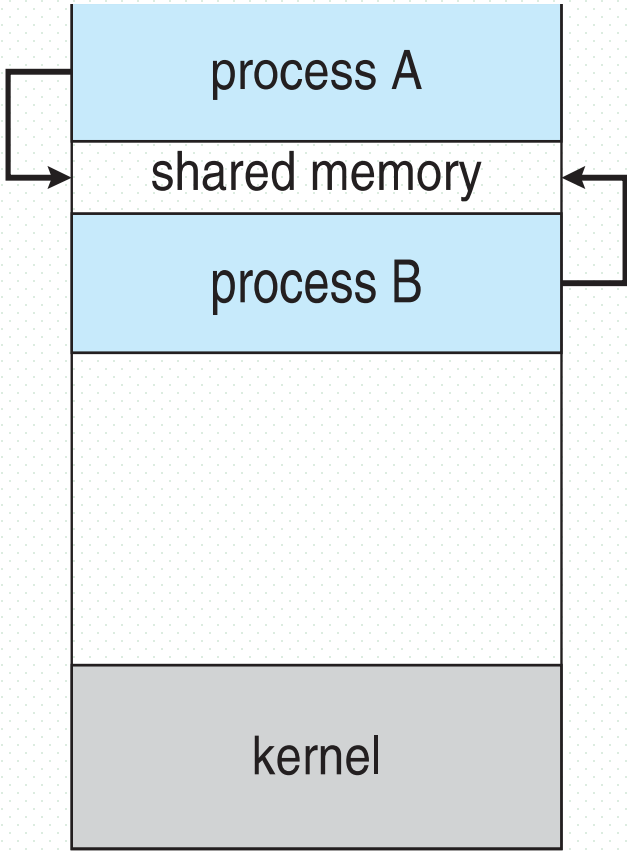
- Cooperating processes need **interprocess communication** (**IPC**)

- Two models of IPC
  - **Shared memory [in user mode]**
  - **Message passing [in kernel mode]**

Linux操作系统实验：进程通信



(a)



(b)

(a) Message passing

(b) shared memory

Example: Message Passing Communication in Unix/Linux

int msgget(key_t, key, int msgflg)

/创建消息队列

int msgsend(int msgid, const void *msg_ptr, size_t msg_sz, int msgflg)

/向消息队列发送消息

int msgrcv(int msgid, void *msg_ptr, size_t msg_st, long int msgtype, int msgflg)

/从消息队列中取消息

Interprocess Communication – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize/同步 their actions when they access shared memory.
- Synchronization is discussed in great details in Chapter 5

Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - send(message)
  - receive(message)

Message Passing Communication in Unix/Linux

```
int msgget(key_t, key, int msgflg)                                          /创建消息队列
int msgsend(int msgid, const void *msg_ptr, size_t msg_sz, int msgflg)      /向消息队列发送消息
int msgrcv(int msgid, void *msg_ptr, size_t msg_st, long int msgtype, int msgflg)  /从消息队列中取消息
```

- The *message* size is either fixed or variable

Message Passing (Cont.)

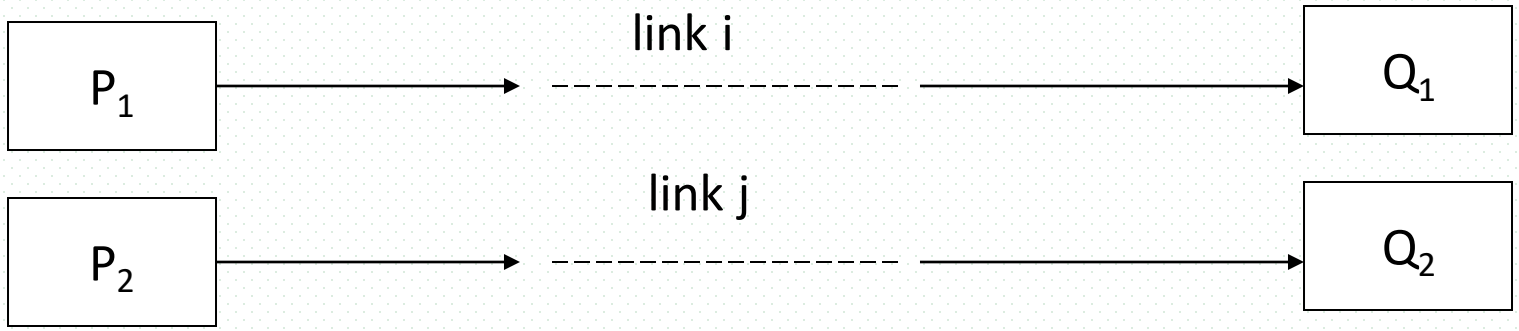
- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?

Message Passing (Cont.)

- Implementation of communication link
  - physical
    - shared memory
    - hardware bus
    - network
  - logical
    - direct or indirect
    - synchronous or asynchronous
    - automatic or explicit buffering

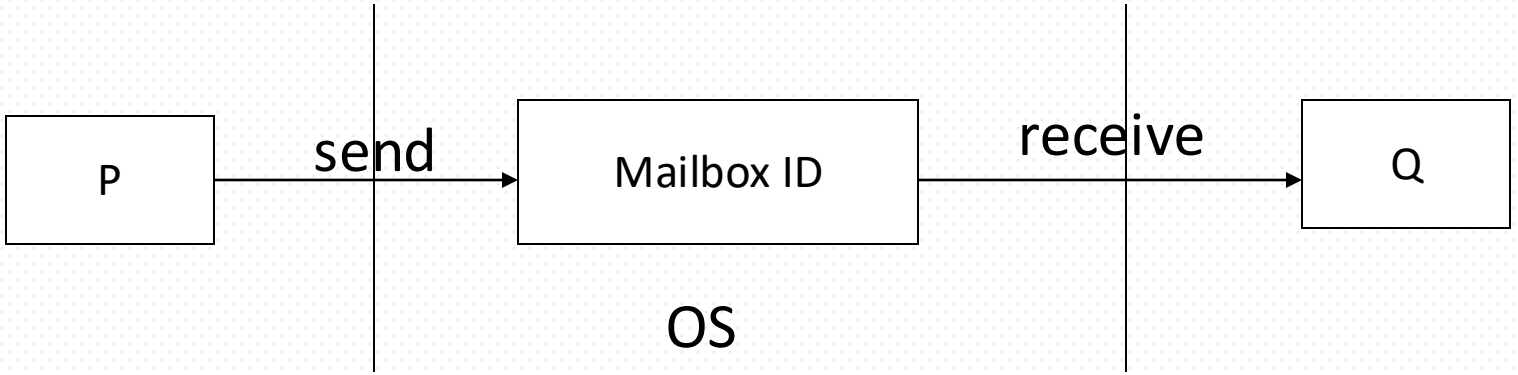
Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*Q, message*) – send a message to process Q
  - **`receive`**(*P, message*) – receive a message from process P
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional



Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id /消息队列
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional



Indirect Communication

- Communication between P and Q
  - P, or Q, or other process ***creates*** a new mailbox ***A***
  - P and Q communicate via ***send***(*mail_**A**, message*) and **receive**(*mail_**A**, message*)
  - when communication is completed, mailbox ***A*** is destroyed
- Primitives are defined as:

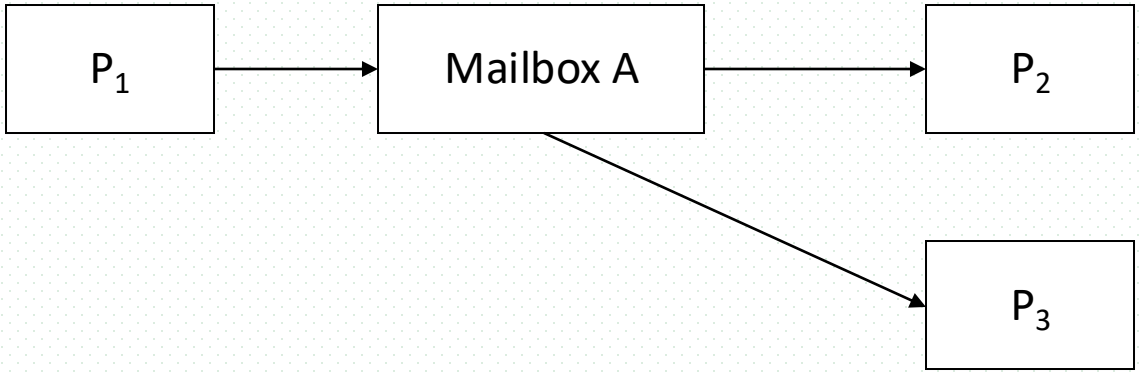
  `send`(*A, message*) – send a message to mailbox A

  `receive`(*A, message*) – receive a message from mailbox A

```
int msgget(key_t, key, int msgflg)                                          /创建消息队列
int msgsend(int msgid, const void *msg_ptr, size_t msg_sz, int msgflg)      /向消息队列发送消息
int msgrcv(int msgid, void *msg_ptr, size_t msg_st, long int msgtype, int msgflg)  /从消息队列中取消息
```

Indirect Communication

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?



- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization among sender/receiver in IPC

- Message passing may be either blocking or non-blocking
- **Blocking/阻塞** is considered **synchronous/同步**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- **Different combinations possible**
  - If both send and receive are blocking, we have a **rendezvous/会合**

3.5 Examples of IPC: POSIX Shared Memory

- POSIX Shared Memory
- Process first creates shared memory segment

```
shm_fd = shm_open(name, O CREAT | O RDWR, 0666);
```

  - Also used to open an existing segment to share it
  - Set the size of the object

```
ftruncate(shm fd, 4096);
```

  - Now the process could write to the shared memory

```
sprintf(shared memory, "Writing to shared memory");
```

Linux操作系统实验：进程通信

IPC POSIX Producer/ Consumer

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

Examples of IPC Systems - Mach

- Mach communication is message based
  - Even system calls are messages
  - Each task gets two mailboxes at creation- Kernel and Notify
  - Only three system calls needed for message transfer

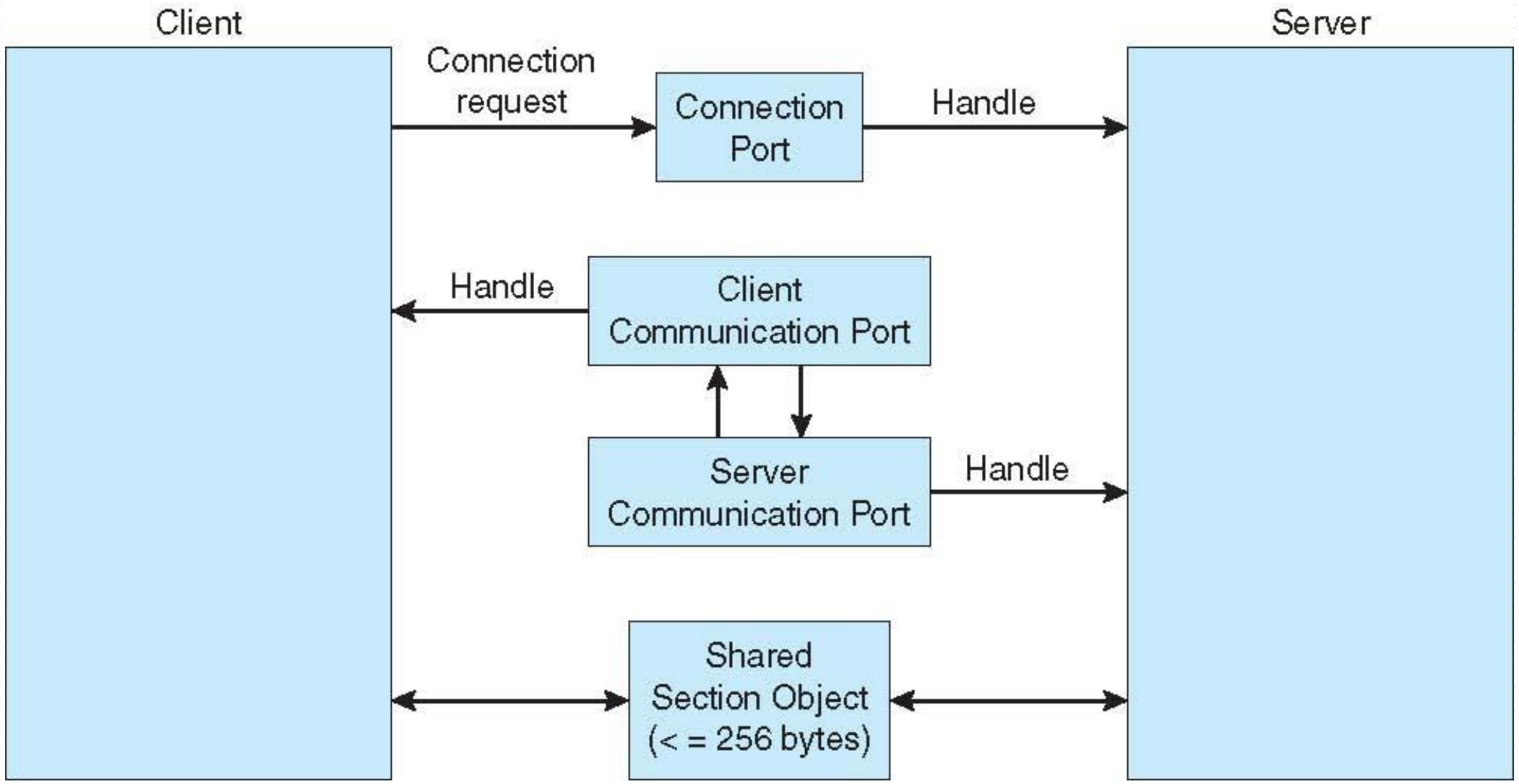
    `msg_send(), msg_receive(), msg_rpc()`
  - Mailboxes needed for commuication, created via

    `port_allocate()`
  - Send and receive are flexible, for example four options if mailbox full:
    - Wait indefinitely
    - Wait at most n milliseconds
    - Return immediately
    - Temporarily cache a message

Examples of IPC Systems – Windows

- Message-passing centric via **advanced local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's **connection port** object.
    - The client sends a connection request.
    - The server creates two private **communication ports** and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies

Local Procedure Calls in Windows

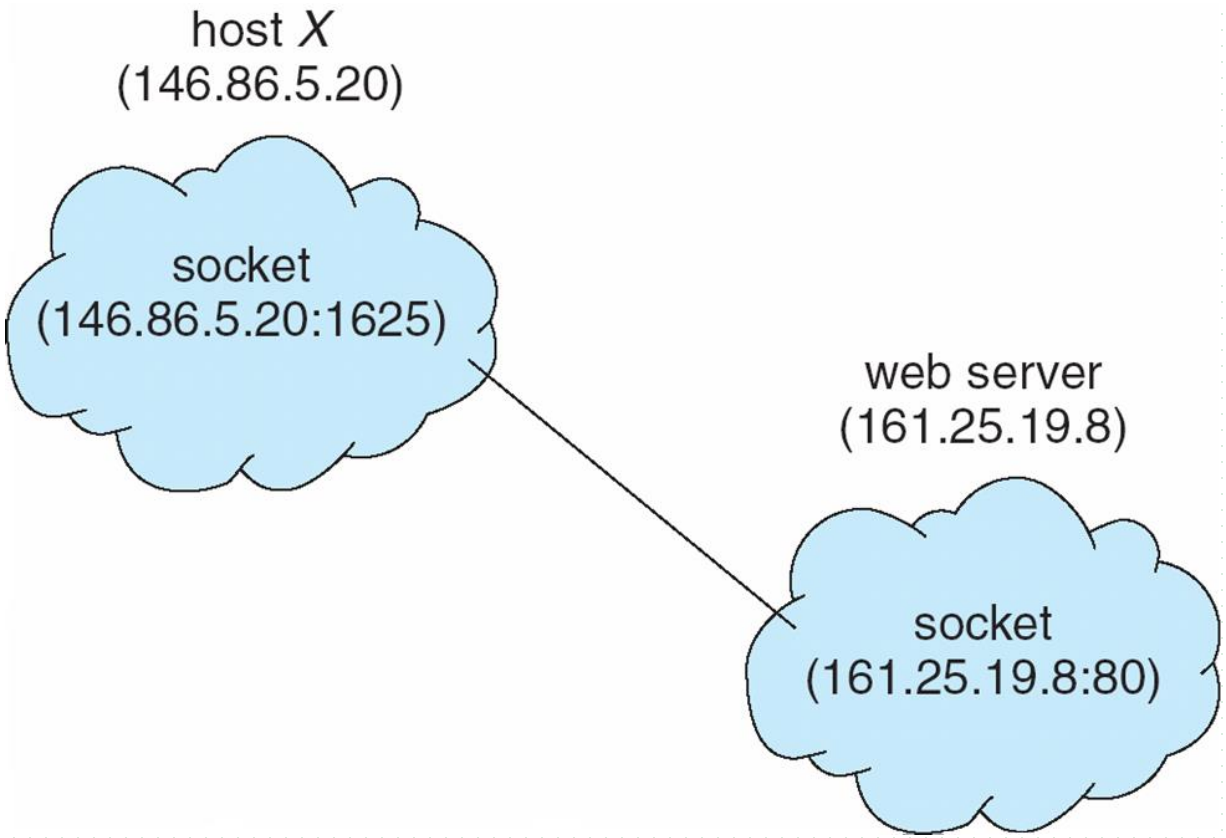


3.6 Communications in Client-Server Systems

- Sockets/套接字
- Remote Procedure Calls
- Pipes/管道
- Remote Method Invocation (Java), 远程方法调用

Sockets

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running



Sockets in Java

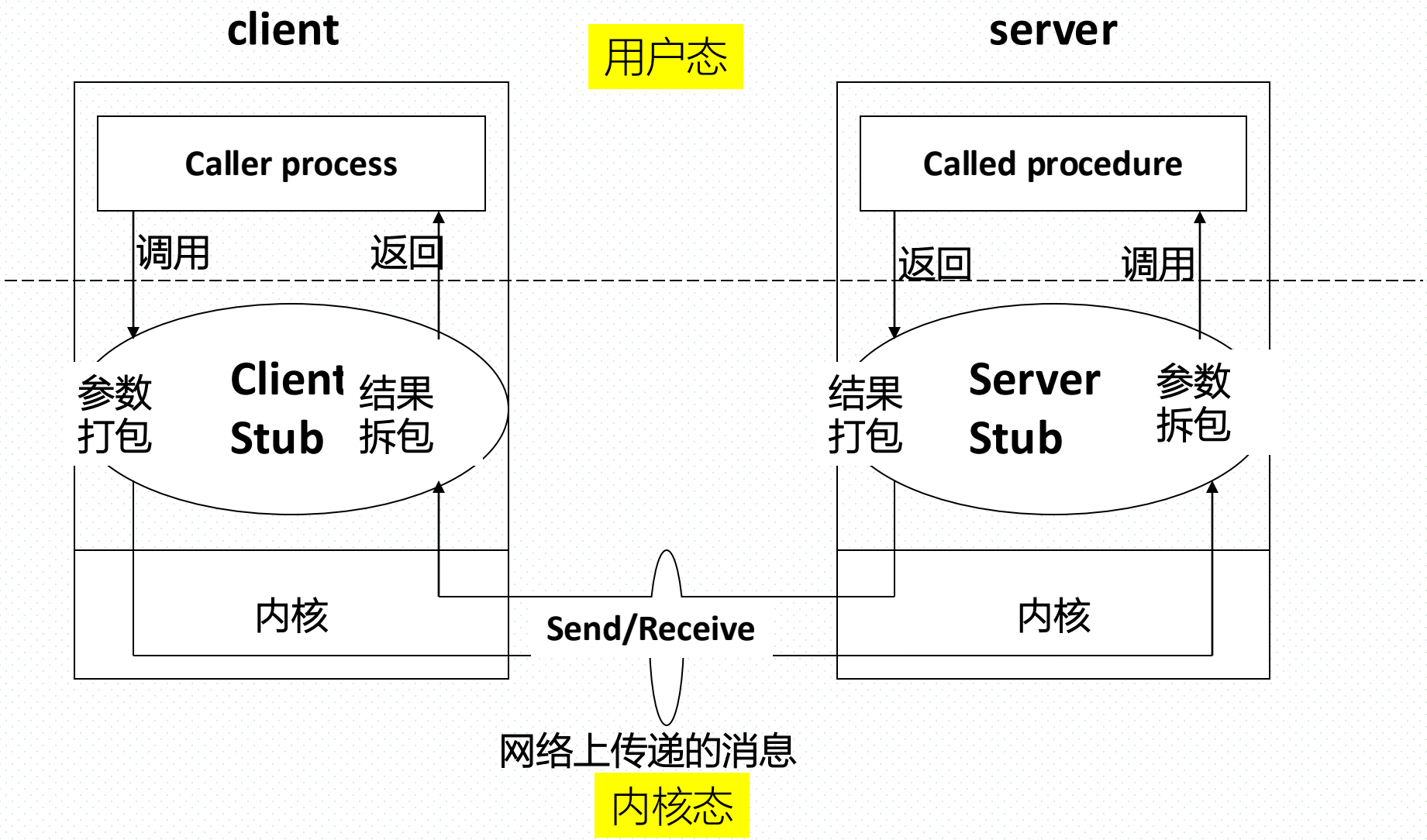
- Three types of sockets
  - **Connection-oriented** (**TCP**)
  - **Connectionless** (**UDP**)
  - **`MulticastSocket`** class– data can be sent to multiple recipients
- Consider this "Date" server:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```

Remote Procedure Calls

- Local procedure call
  - programming in user space/mode, independent on OS
  - needing no system calls, etc.
- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems
  - Again uses ports for service differentiation
- **Stubs/存根** – client-side proxy for the actual procedure on the server
- The client-side stub locates the server and **marshalls/封装** the parameters
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server
- On Windows, stub code compile from specification written in **Microsoft Interface Definition Language** (**MIDL**)

Remote Procedure Calls



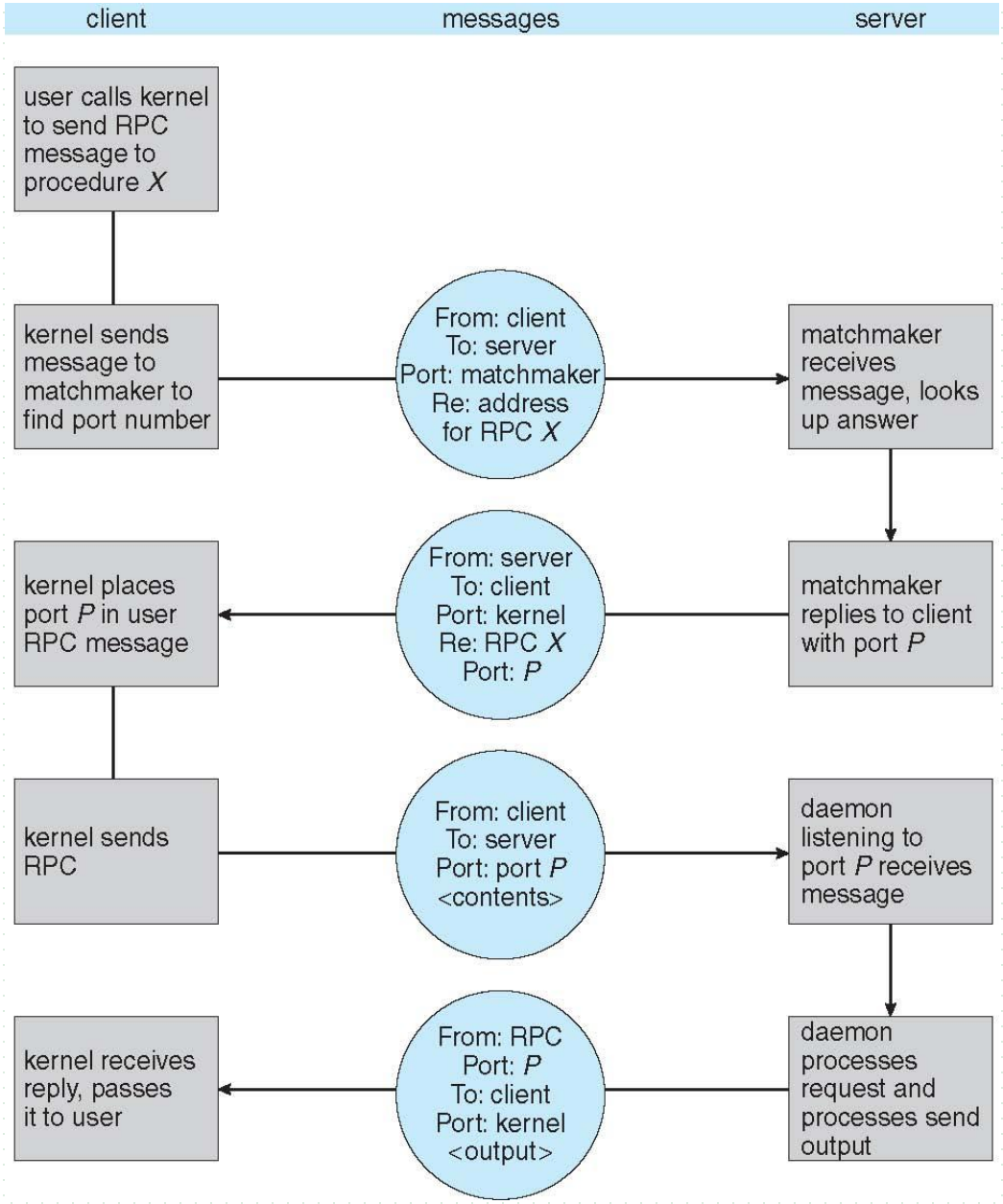
Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation** (**XDL**) format to account for different architectures
  - **big-endian** and **little-endian/大端结尾、小端结尾**
- Remote communication has more failure scenarios than local
  - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server

RPC过程

- 客户程序（caller process）按通常的（类似于本地的）调用方式，调用客户存根
- 客户存根创建一个消息，封装参数，并陷入内核
- 内核将该消息发送给服务器端内核
- 服务器端内核将该消息传递给服务器存根
- 服务器存根从消息中获取参数，并调用服务器程序（called process）
- 服务器程序完成工作，将结果返回给服务器存根
- 服务器存根将结果打包进消息，并陷入OS内核。
- 服务器内核将消息返回给客户端内核
- 客户端内核将消息传递给客户存根
- 客户存根取出结果，返回给客户端调用程序

Remote Procedure Calls (Cont.)

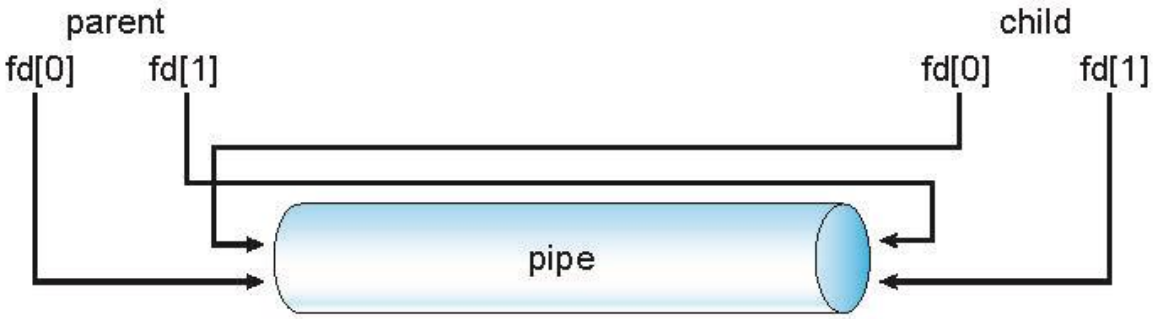


Pipes

- Acts as a conduit allowing two processes to communicate
- Issues:
  - Is communication unidirectional or bidirectional?
  - In the case of two-way communication, is it half or full-duplex?
  - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
  - Can the pipes be used over a network?
- Ordinary pipes – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- Named pipes – can be accessed without a parent-child relationship.

Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**
- See Unix and Windows code samples in textbook

Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems

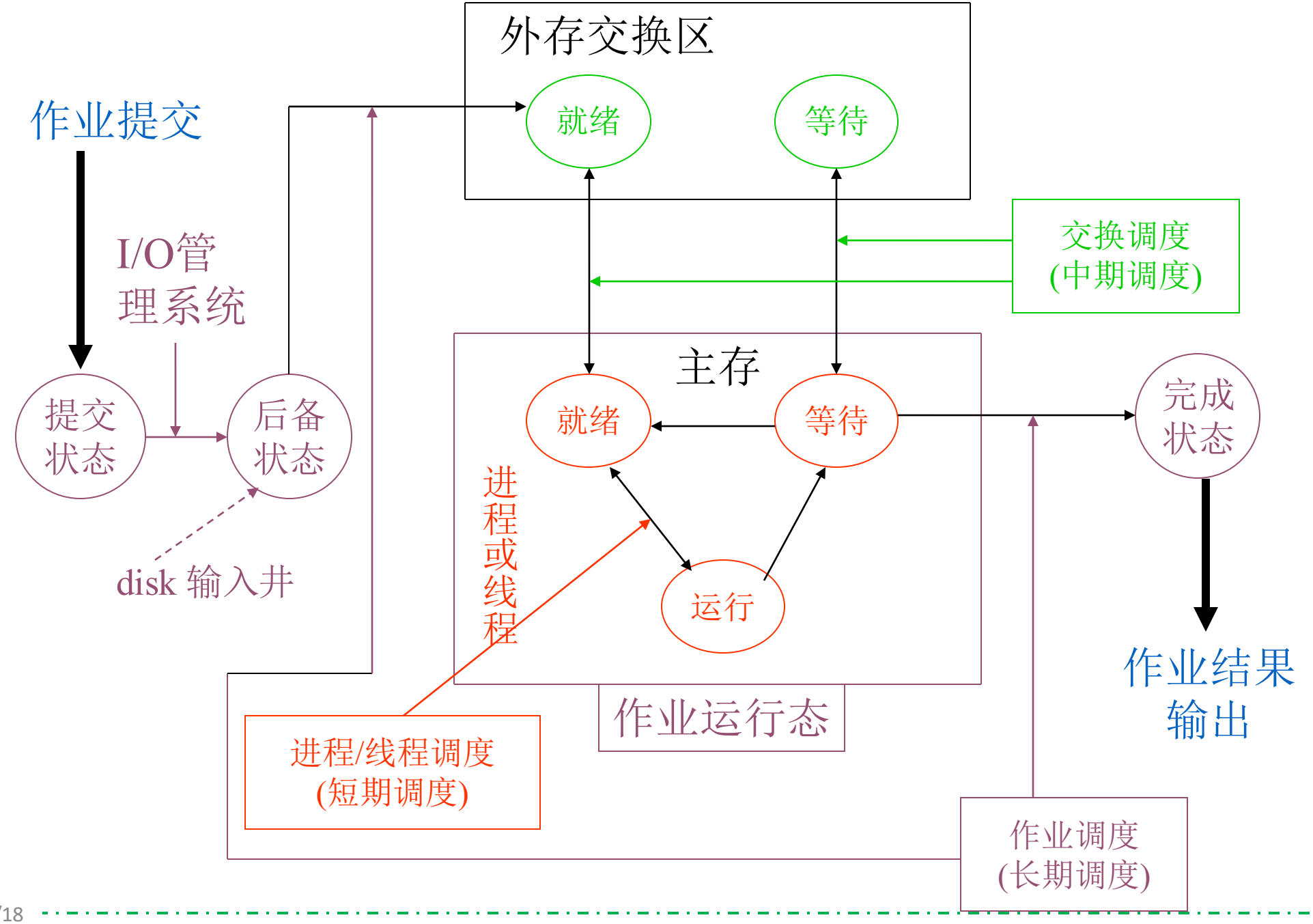
Appendix 3-1 调度的层次与作业调度

- 在大型通用系统和工作站中（多道批处理系统，分时系统），用户向系统提交作业，请求系统的服务
- 作业与进程的关系：
  - 作业是用户向计算机提交任务的任务实体
  - 进程/线程是OS为完成用户任务实体而设置的执行实体，是资源分配与处理器调度的基本单位
  - 一个作业可对应于多个执行进程/线程（根进程—子进程）

- 作业从进入系统到最终完成经过3个阶段
  - 作业通过I/O通道进入外部存储设备（磁盘，磁带机）上 的输入井
  - 长期调度程序调度作业进入主存，以进程/线程状态存在.
  - 进程/线程获得CPU，并运行，直至作业完成

调度的层次与作业调度

- 作业在整个生命周期过程中可划分为4个状态
  - 提交态：作业处于输入系统（提交）过程中
  - 后备/收容态：作业全部进入系统，处于外设输入井中等待运行
  - 运行态：作业被作业调度程序选中，进入主存，OS为其创建进程/线程并投入运行
  - 完成态：作业完成全部运行，释放所占用全部资源，准备退出系统。


外存交换区
就绪
等待
作业提交
I/O管
理系统
交换调度
(中期调度)
主存
提交
状态
后备
状态
就绪
等待
完成
状态
进
程
或
线
程
disk 输入井
运行
作业结果
输出
作业运行态
进程/线程调度
(短期调度)
作业调度
(长期调度)

调度的层次与作业调度

- 作业调度
  - 又称为长期调度、宏观调度、高级调度
  - 按照一定原则从输入井中的磁盘队列中选取作业进入主存，并分配必要资源。时间上通常是分钟、小时或天。
- 进程或线程调度
  - 又称为“微观调度”、“低级调度”。从CPU资源的角度，执行的基本单位的调度。时间上通常是毫秒。因为执行频繁，要求在实现时达到高效率。
- （内外存）交换调度
  - 又称为中期调度。将处于就绪态或等待态的某些进程在主存和外存交换区中倒换，以保证主存使用效率和进程执行效率
  - 属于内存管理与扩充范畴

Thanks for your attention

北京郵電大學